

ets_marl - Thesis Experiments Analysis

May 8, 2026

1 ETS MARL — Thesis Experiments Analysis

This notebook analyzes the thesis experiment sweep against the MAIN/default seed sweep. It is designed as the cross-config counterpart to `ets_marl - Default RQ Analysis.ipynb` and answers the RQ2 sensitivity questions while reusing MAIN for RQ1/RQ3 decomposition.

Inputs expected

- MAIN/default seeds from `configs/sweeps/default_seeds.yaml` (`results/sweeps/default/` or `results/sweeps/default/default/`, depending on where outputs were moved).
- Thesis variants from `configs/sweeps/thesis_experiments.yaml` (`results/sweeps/thesis_experiments/<variant>/`).
- Same paired seeds for MAIN and the structural variants: 1729, 8191, 6561. MAIN may also include seed 5041; paired comparisons filter to common seeds where needed.

Variants covered

- MAIN — default LRF/MSR/objective mix.
- `lrf_low_22pct` — lower LRF (`lrf_low`, 2.2%).
- `lrf_high_6pct` — aggressive LRF (`lrf_high`, 6.3% in config; labelled 6% in thesis shorthand).
- `no_market_stability` — `no_msr`, with MSR and unsold-to-MSR disabled.
- `all_financial_objectives` — `all_fin`, all learning agents pure financial.
- `balanced_identical` — `balanced`, identical endowments with 4 financial + 4 ESG agents.
- `high_inflation` — `high_inf`, single-seed probe.

Run all cells after the sweep finishes. If some runs are missing, the notebook will load what exists and mark affected comparisons explicitly.

```
[1]: # Install notebook dependencies if needed.
# Set INSTALL_NOTEBOOK_DEPS = True to force-install the shared notebook
# requirements.
import subprocess, sys, importlib, re
from pathlib import Path

INSTALL_NOTEBOOK_DEPS = False
_DIST_TO_IMPORT = {
    'pyyaml': 'yaml',
    'scikit-learn': 'sklearn',
```

```

    'imbalanced-learn': 'imblearn',
    'opencv-python': 'cv2',
    'pillow': 'PIL',
    'ipython': 'IPython',
}

def _find_requirements():
    req_name = 'requirements-notebooks.txt'
    here = Path.cwd()
    candidates = [here / req_name, here / 'notebooks' / req_name]
    candidates.extend(p / 'notebooks' / req_name for p in here.parents)
    return next((p for p in candidates if p.exists()), None)

def _parse_requirements(req_path):
    dists = []
    for raw in req_path.read_text(encoding='utf-8').splitlines():
        line = raw.strip()
        if not line or line.startswith('#') or line.startswith('-'):
            continue
        line = line.split('; ', 1)[0].strip()
        m = re.match(r'^([A-Za-z0-9_.\-]+)', line)
        if m:
            dists.append(m.group(1).lower())
    return dists

def _missing_required(dists):
    missing = []
    for dist in dists:
        mod = _DIST_TO_IMPORT.get(dist, dist.replace('-', '_'))
        try:
            importlib.import_module(mod)
        except ImportError:
            missing.append(dist)
    return missing

req_path = _find_requirements()
if req_path is None:
    raise FileNotFoundError('Could not find notebooks/requirements-notebooks.
↳txt')
missing = _missing_required(_parse_requirements(req_path))
if INSTALL_NOTEBOOK_DEPS or missing:
    if missing:
        print(f'Missing packages: {missing}')
    print(f'Installing notebook dependencies from: {req_path}')
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-r',
↳str(req_path)])
else:

```

```
print('Notebook dependencies available; skipping install.')
```

Notebook dependencies available; skipping install.

1.1 1. Setup — Locate Project Root

```
[2]: import os, sys
from pathlib import Path

def _is_project_root(path):
    path = Path(path)
    return (path / 'src').is_dir() and (path / 'configs').is_dir()

env_root = os.environ.get('ETS_MARL_PROJECT_ROOT') or os.environ.
    get('PROJECT_ROOT')
candidates = [Path.cwd(), Path.cwd().parent, Path.cwd().parent.parent]
if env_root:
    candidates.append(Path(env_root))
PROJECT_ROOT = next((p.resolve() for p in candidates if _is_project_root(p)),
    None)
if PROJECT_ROOT is None:
    raise FileNotFoundError(f'Cannot find project root from {Path.cwd()}')
os.chdir(PROJECT_ROOT)
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
print(f'Working directory: {PROJECT_ROOT}')
```

Working directory: C:\Users\danie\Documents\Python_Scripts\Master Thesis\Thesis-Energy-Auction

1.2 2. Imports

```
[3]: import copy
import math
import re
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import yaml
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm.auto import tqdm

from src.utils.run_data import load_run_csv, log_exists
from src.utils.sweep import load_sweep_spec, expand_dotted_overrides, deep_merge
```

```

sns.set_style('whitegrid')
pd.set_option('display.max_columns', 240)
pd.set_option('display.width', 220)

EPS = 1e-9

```

1.3 3. Sweep Specs, Variant Labels, and Discovery

```

[4]: MAIN_SPEC = PROJECT_ROOT / 'configs' / 'sweeps' / 'default_seeds.yaml'
EXPERIMENT_SPEC = PROJECT_ROOT / 'configs' / 'sweeps' / 'thesis_experiments.
    ↪yaml'

# Leave as None to use spec.output_dir. Add paths here if outputs were moved
↪after the run.
OUTPUT_DIR_OVERRIDES = {
    'default_seeds': None,
    'thesis_experiments': None,
}

DISPLAY_NAME = {
    'default': 'MAIN',
    'lrf_low': 'lrf_low_22pct',
    # Fixed thesis shorthand label; later analysis derives the exact LRF
    # percentage from the loaded config rather than from this display string.
    'lrf_high': 'lrf_high_6pct',
    'no_msr': 'no_market_stability',
    'all_fin': 'all_financial_objectives',
    'balanced': 'balanced_identical',
    'high_inf': 'high_inflation',
}

VARIANT_ORDER = ['MAIN', 'lrf_low_22pct', 'lrf_high_6pct',
    ↪'no_market_stability',
    ↪'all_financial_objectives', 'balanced_identical',
    ↪'high_inflation']
PAIRED_SEEDS = [1729, 8191, 6561]

SPEC_PATHS = [MAIN_SPEC, EXPERIMENT_SPEC]
for sp in SPEC_PATHS:
    if not sp.exists():
        warnings.warn(f'Sweep spec not found: {sp}')

```

```

[5]: def _flatten_dict(d, prefix=''):
    out = {}
    if not isinstance(d, dict):
        return out

```

```

for k, v in d.items():
    key = f'{prefix}.{k}' if prefix else str(k)
    if isinstance(v, dict):
        out.update(_flatten_dict(v, key))
    else:
        out[key] = v
return out

def _variant_display(raw_name):
    return DISPLAY_NAME.get(raw_name, raw_name)

def _as_path_from_spec(spec, spec_path, override=None):
    if override is not None:
        return Path(override).expanduser().resolve()
    p = Path(spec['output_dir'])
    return p if p.is_absolute() else (PROJECT_ROOT / p).resolve()

def _candidate_dirs(output_dir, variant_name, display_name, spec_name):
    dirs = []
    def add(p):
        p = Path(p)
        if p not in dirs:
            dirs.append(p)
    add(output_dir / variant_name)
    add(output_dir / display_name)
    add(output_dir)
    # Common locations used by earlier notebooks / moved sweep outputs.
    add(PROJECT_ROOT / 'results' / 'sweeps' / variant_name)
    add(PROJECT_ROOT / 'results' / 'sweeps' / display_name)
    if display_name == 'MAIN':
        add(PROJECT_ROOT / 'results' / 'sweeps' / 'default')
        add(PROJECT_ROOT / 'results' / 'sweeps' / 'default' / 'default')
    add(PROJECT_ROOT / 'results' / 'sweep' / spec_name / variant_name)
    add(PROJECT_ROOT / 'results' / 'sweep' / spec_name)
    return dirs

def _resolve_run_files(variant_name, display_name, seed, output_dir, spec_name):
    training_names = [
        f'training_log_{variant_name}_s{seed}.csv',
        f'training_log_{display_name}_s{seed}.csv',
        f'training_log_s{seed}.csv',
    ]
    year_names = [
        f'year_log_{variant_name}_s{seed}.csv',
        f'year_log_{display_name}_s{seed}.csv',
        f'year_log_s{seed}.csv',
    ]

```

```

    tlog = ylog = None
    for base in _candidate_dirs(output_dir, variant_name, display_name,
↪spec_name):
        if tlog is None:
            for name in training_names:
                cand = base / name
                if log_exists(cand):
                    tlog = cand
                    break
        if ylog is None:
            for name in year_names:
                cand = base / name
                if log_exists(cand):
                    ylog = cand
                    break
        if tlog is not None and ylog is not None:
            break
    return tlog, ylog

# Column subsets keep notebook memory manageable. The shared helper in
# src.utils.run_data is the single source of truth across analysis
# notebooks; unknown columns are silently dropped by load_run_csv.
from src.utils.run_data import default_analysis_columns
EP_COLUMNS, YR_COLUMNS = default_analysis_columns(16)

# Tail-window load size. The notebook only consumes the converged tail
# (TAIL_EPISODES=200 in cell 11 via _tail_ep / _tail_yr); loading just
# the last YR_TAIL_EPISODES episodes per run keeps 16-20 cell sweeps
# in RAM. Set to None below to disable and load full training history.
YR_TAIL_EPISODES = 1000

```

```

[7]: from IPython.display import display as ipy_display

specs = []
for spec_path in SPEC_PATHS:
    if not spec_path.exists():
        continue
    spec_name = spec_path.stem
    spec = load_sweep_spec(str(spec_path))
    with open(spec['base_config'], encoding='utf-8') as f:
        base_cfg = yaml.safe_load(f)
    output_dir = _as_path_from_spec(spec, spec_path, OUTPUT_DIR_OVERRIDES.
↪get(spec_name))
    specs.append({'name': spec_name, 'path': spec_path, 'spec': spec,
↪'base_cfg': base_cfg, 'output_dir': output_dir})

load_jobs = []

```

```

for s in specs:
    for v in s['spec'].get('variants', []):
        raw_variant = v['name']
        display = _variant_display(raw_variant)
        seeds = v.get('seeds') if v.get('seeds') is not None else s['spec'].
        ↪get('seeds', [])
        overrides = expand_dotted_overrides(v.get('overrides') or {})
        merged_cfg = deep_merge(copy.deepcopy(s['base_cfg']), overrides)
        for seed in seeds:
            load_jobs.append((s, v, display, seed, merged_cfg, overrides))

runs = {}
discovery = []
for s, v, display, seed, merged_cfg, overrides in tqdm(load_jobs,
    ↪desc='Discovering/loading runs', unit='run'):
    raw_variant = v['name']
    tlog, ylog = _resolve_run_files(raw_variant, display, seed,
    ↪s['output_dir'], s['name'])
    row = {
        'sweep': s['name'], 'variant_raw': raw_variant, 'variant': display,
    ↪'seed': seed,
        'has_overrides': bool(overrides), 'training_log': str(tlog) if tlog
    ↪else None,
        'year_log': str(ylog) if ylog else None, 'has_training_log': tlog is
    ↪not None,
        'has_year_log': ylog is not None, 'found': (tlog is not None) or (ylog
    ↪is not None),
    }
    discovery.append(row)
    if not row['found']:
        continue
    # Cheap pre-scan: read only the 'episode' column to record total
    # training length before tail-loading the rest. This preserves
    # `len(ep_df)`-style semantics for diagnostics that report total
    # episodes, while keeping peak RAM bounded by YR_TAIL_EPISODES.
    n_total_episodes = 0
    for _meta_path in (tlog, ylog):
        if _meta_path is None:
            continue
        _ep_only = load_run_csv(_meta_path, columns=['episode'])
        if _ep_only is not None and not _ep_only.empty and 'episode' in
    ↪_ep_only.columns:
            n_total_episodes = int(_ep_only['episode'].max()) + 1
            break
    del _ep_only

```

```

    ep_df = load_run_csv(tlog, columns=EP_COLUMNS,
    ↪tail_episodes=YR_TAIL_EPISODES) if tlog is not None else None
    yr_df = load_run_csv(ylog, columns=YR_COLUMNS,
    ↪tail_episodes=YR_TAIL_EPISODES) if ylog is not None else None
    if ep_df is None and yr_df is None:
        continue
    if ep_df is None:
        ep_df = pd.DataFrame({'episode': np.sort(yr_df['episode'].dropna().
    ↪unique())}) if 'episode' in yr_df.columns else pd.DataFrame()
    if yr_df is None:
        yr_df = pd.DataFrame({'episode': ep_df['episode'].values, 'year': np.
    ↪zeros(len(ep_df), dtype=int)}) if 'episode' in ep_df.columns else pd.
    ↪DataFrame()
    runs[(display, int(seed))] = {
        'sweep': s['name'], 'variant_raw': raw_variant, 'variant': display,
    ↪'seed': int(seed),
        'ep_df': ep_df, 'yr_df': yr_df, 'config': merged_cfg, 'overrides':
    ↪overrides,
        'training_log': str(tlog) if tlog else None, 'year_log': str(ylog) if
    ↪ylog else None,
        'n_total_episodes': n_total_episodes if n_total_episodes else
    ↪len(ep_df),
    }
    # Release pandas/pyarrow buffers between runs so 16-20 cell sweeps don't
    # accumulate a permanent RAM high-water mark.
    import gc; gc.collect()

discovery_df = pd.DataFrame(discovery)
print(f'Discovered {len(discovery_df)} declared runs; loaded {len(runs)}
    ↪available runs.')
if not discovery_df.empty:
    ipy_display(
        discovery_df.sort_values(['variant', 'seed'])[
            ['variant', 'seed', 'found', 'has_training_log', 'has_year_log',
    ↪'training_log', 'year_log']
        ]
    )
if not runs:
    raise RuntimeError('No runs loaded. Check output directories above or set
    ↪OUTPUT_DIR_OVERRIDES.')

# Free chunked CSV/parquet readers held by pandas/pyarrow before continuing.
import gc; gc.collect()

```

Discovering/loading runs: 0% | 0/20 [00:00<?, ?run/s]

Discovered 20 declared runs; loaded 20 available runs.

	variant	seed	found	has_training_log	has_year_log	
			training_log			
	year_log					
0	MAIN	1729	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
3	MAIN	5041	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
2	MAIN	6561	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
1	MAIN	8191	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
13	all_financial_objectives	1729	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
15	all_financial_objectives	6561	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
14	all_financial_objectives	8191	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
16	balanced_identical	1729	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
18	balanced_identical	6561	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
17	balanced_identical	8191	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
19	high_inflation	1729	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
7	lrf_high_6pct	1729	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
9	lrf_high_6pct	6561	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					
8	lrf_high_6pct	8191	True	True	True	C:
	\Users\danie\Documents\Python_Scripts\Master...					C:
	\Users\danie\Documents\Python_Scripts\Master...					

```

4         lrf_low_22pct  1729  True                True                True  C:
   ↪\Users\danie\Documents\Python_Scripts\Master... C:
   ↪\Users\danie\Documents\Python_Scripts\Master...
6         lrf_low_22pct  6561  True                True                True  C:
   ↪\Users\danie\Documents\Python_Scripts\Master... C:
   ↪\Users\danie\Documents\Python_Scripts\Master...
5         lrf_low_22pct  8191  True                True                True  C:
   ↪\Users\danie\Documents\Python_Scripts\Master... C:
   ↪\Users\danie\Documents\Python_Scripts\Master...
10        no_market_stability 1729  True                True                True  C:
   ↪\Users\danie\Documents\Python_Scripts\Master... C:
   ↪\Users\danie\Documents\Python_Scripts\Master...
12        no_market_stability 6561  True                True                True  C:
   ↪\Users\danie\Documents\Python_Scripts\Master... C:
   ↪\Users\danie\Documents\Python_Scripts\Master...
11        no_market_stability 8191  True                True                True  C:
   ↪\Users\danie\Documents\Python_Scripts\Master... C:
   ↪\Users\danie\Documents\Python_Scripts\Master...

```

[7]: 19

1.4 4. Shared Helpers

```

[8]: REFERENCE_CONFIG = next(iter(runs.values()))['config']
N_AGENTS = int(REFERENCE_CONFIG.get('companies', {}).get('n_agents', 8))
N_BOTS = int(REFERENCE_CONFIG.get('companies', {}).get('n_bot_agents', 0))
N_TOTAL = N_AGENTS + N_BOTS
N_YEARS = int(REFERENCE_CONFIG.get('simulation', {}).get('n_years', 12)
   ↪REFERENCE_CONFIG.get('environment', {}).get('n_years', 12)))
TAIL_EPISODES = 200
ROLL = 50

ARCHETYPES = ['Coal-heavy', 'Gas-dominant', 'Transitioner', 'Green-leader']

def agent_cols(df, prefix, max_idx=N_AGENTS):
    rx = re.compile(rf'~{re.escape(prefix)}_A(\d+)$')
    cols = []
    for c in df.columns:
        m = rx.match(c)
        if m and int(m.group(1)) <= max_idx:
            cols.append((int(m.group(1)), c))
    return [c for _, c in sorted(cols)]

def _agent_archetype(idx0):
    return ARCHETYPES[idx0 // 2] if idx0 // 2 < len(ARCHETYPES) else
   ↪f'Custom-{idx0 // 2}'

```

```

def _reward_weights_for(run):
    return run['config'].get('companies', {}).get('reward_weights', [])

def _agent_weighting(run, idx0):
    weights = _reward_weights_for(run)
    if idx0 < len(weights):
        w_cost, w_green = weights[idx0]
        if float(w_green) <= 1e-8:
            return 'Financial'
        if abs(float(w_cost) - 0.5) < 1e-6 and abs(float(w_green) - 0.5) < 1e-6:
            return 'Balanced ESG'
        return f'w=({w_cost:g},{w_green:g})'
    return 'Financial' if idx0 % 2 == 0 else 'Balanced ESG'

def _tail_ep(run, n=TAIL_EPISODES):
    ep = run['ep_df']
    return ep.tail(min(n, len(ep))) if len(ep) else ep

def _tail_yr(run, n=TAIL_EPISODES):
    yr = run['yr_df']
    ep = _tail_ep(run, n)
    if yr.empty or ep.empty or 'episode' not in yr.columns or 'episode' not in ep.columns:
        return yr.head(0)
    return yr[yr['episode'].isin(ep['episode'].dropna().unique())]

def _yearT(yr):
    if yr.empty or 'year' not in yr.columns:
        return yr.head(0)
    return yr[yr['year'] == yr['year'].max()]

def _safe_mean(x):
    if not isinstance(x, pd.Series):
        x = pd.Series(x)
    x = x.replace([np.inf, -np.inf], np.nan).dropna()
    return float(x.mean()) if len(x) else np.nan

def _safe_std(x):
    if not isinstance(x, pd.Series):
        x = pd.Series(x)
    x = x.replace([np.inf, -np.inf], np.nan).dropna()
    return float(x.std()) if len(x) > 1 else np.nan

def _gini(values):
    arr = np.asarray(values, dtype=float)
    arr = arr[np.isfinite(arr)]
    if arr.size == 0:

```

```

        return np.nan
    if np.all(arr <= 0):
        return 0.0
    arr = np.sort(np.clip(arr, 0, None))
    n = arr.size
    return float((2 * np.arange(1, n + 1) @ arr) / (n * arr.sum()) - (n + 1) /
↪n)

def _rowwise_gini(df, cols):
    if not cols:
        return pd.Series(dtype=float)
    return df[cols].apply(lambda r: _gini(r.values), axis=1)

def _floor_price(run):
    cfg = run['config']
    return float(cfg.get('auction', {}).get('price_min', cfg.get('ets', {})).
↪get('reserve_price', 45.0)))

def _ceiling_price(run):
    return float(run['config'].get('auction', {}).get('price_max', 250.0))

def _variant_color_map():
    palette = sns.color_palette('tab10', n_colors=max(len(VARIANT_ORDER), 3))
    return {name: palette[i % len(palette)] for i, name in
↪enumerate(VARIANT_ORDER)}
VARIANT_COLOR = _variant_color_map()

def paired_runs(variants, seeds=PAIRED_SEEDS):
    return {k: v for k, v in runs.items() if k[0] in variants and k[1] in seeds}

def _ordered_variants_present():
    present = [v for v in VARIANT_ORDER if any(k[0] == v for k in runs)]
    extras = sorted({k[0] for k in runs} - set(present))
    return present + extras

print(f'Reference shape: N_AGENTS={N_AGENTS}, N_BOTS={N_BOTS},
↪N_YEARS={N_YEARS}, tail={TAIL_EPISODES} episodes')
print('Loaded variants:', _ordered_variants_present())

```

```

Reference shape: N_AGENTS=8, N_BOTS=0, N_YEARS=12, tail=200 episodes
Loaded variants: ['MAIN', 'lrf_low_22pct', 'lrf_high_6pct',
'no_market_stability', 'all_financial_objectives', 'balanced_identical',
'high_inflation']

```

```

[10]: def summarize_run(run, n=TAIL_EPISODES):
        ep = _tail_ep(run, n)
        yr = _tail_yr(run, n)

```

```

yT = _yearT(yr)
price_col = 'clearing_price' if 'clearing_price' in yr.columns else None
floor = _floor_price(run)
ceil = _ceiling_price(run)

gf_cols_yr = agent_cols(yr, 'green_frac')
gf_cols_ep = agent_cols(ep, 'green_frac')
sf_cols = agent_cols(yr, 'shortfall')
pen_cols = agent_cols(yr, 'penalty')
bank_cols = agent_cols(yr, 'bank_end')
cf_cols = agent_cols(yr, 'carry_forward_end')
alloc_cols = agent_cols(yr, 'alloc')
inv_cols_yr = agent_cols(yr, 'invest_cost')
bid_cols = agent_cols(yr, 'bid_price')
invest_frac_cols = agent_cols(yr, 'invest_frac_post_clip')
pre_cols = agent_cols(yr, 'invest_frac_pre_clip')
post_cols = agent_cols(yr, 'invest_frac_post_clip')

penalty_values = yr[pen_cols].to_numpy().ravel() if pen_cols else np.
↳array([])
short_values = yr[sf_cols].to_numpy().ravel() if sf_cols else np.array([])
if alloc_cols:
    gini_s = _rowwise_gini(yr, alloc_cols)
elif 'auction_hhi' in yr.columns:
    gini_s = yr['auction_hhi'] / 10000.0
else:
    gini_s = pd.Series(dtype=float)

row = {
    'variant': run['variant'], 'seed': run['seed'], 'n_episodes': run.
↳get('n_total_episodes', len(run['ep_df'])),
    'tail_episodes': len(ep),
    'mean_price': _safe_mean(yr[price_col]) if price_col else _safe_mean(ep.
↳get('ep_mean_clearing_price', pd.Series(dtype=float))),
    'final_year_price': _safe_mean(yT[price_col]) if price_col and not yT.
↳empty else _safe_mean(ep.get('clearing_price_last', pd.Series(dtype=float))),
    'within_episode_price_std': _safe_mean(yr.groupby('episode')[price_col].
↳std()) if price_col and 'episode' in yr.columns else _safe_mean(ep.
↳get('price_std', pd.Series(dtype=float))),
    'floor_touch_freq': float((yr[price_col] <= floor * 1.01).mean()) if
↳price_col and len(yr) else np.nan,
    'ceiling_touch_freq': float((yr[price_col] >= ceil * 0.99).mean()) if
↳price_col and len(yr) else np.nan,
    'green_pct_year12': 100.0 * _safe_mean(yT[gf_cols_yr].mean(axis=1)) if
↳gf_cols_yr and not yT.empty else 100.0 * _safe_mean(ep[gf_cols_ep].
↳mean(axis=1)) if gf_cols_ep else np.nan,

```

```

    'green_pct_all_years': 100.0 * _safe_mean(yr[gf_cols_yr].mean(axis=1))
    ↪if gf_cols_yr else np.nan,
    'investment_total': _safe_mean(yr[inv_cols_yr].sum(axis=1)) if
    ↪inv_cols_yr else _safe_mean(ep[[c for c in agent_cols(ep, 'invest_cost')]].
    ↪sum(axis=1)) if agent_cols(ep, 'invest_cost') else np.nan,
    'compliance_rate': float((short_values <= 1e-6).mean()) if short_values.
    ↪size else np.nan,
    'penalty_frequency': float((penalty_values > 1e-6).mean()) if
    ↪penalty_values.size else np.nan,
    'penalty_magnitude_positive': float(penalty_values[penalty_values >
    ↪1e-6].mean()) if np.any(penalty_values > 1e-6) else 0.0,
    'penalty_total': float(np.nansum(penalty_values) / max(1, yr['episode'].
    ↪nunique())) if penalty_values.size and 'episode' in yr.columns else np.nan,
    'tnac_year12': _safe_mean(yT['tnac']) if 'tnac' in yT.columns and not
    ↪yT.empty else _safe_mean(ep.get('yearT_tnac', pd.Series(dtype=float))),
    'tnac_mean': _safe_mean(yr['tnac']) if 'tnac' in yr.columns else np.nan,
    'msr_withhold_events': int((yr.get('msr_withhold_this_year', pd.
    ↪Series(dtype=float)) > 1e-6).sum()) if len(yr) else 0,
    'msr_release_events': int((yr.get('msr_release_this_year', pd.
    ↪Series(dtype=float)) > 1e-6).sum()) if len(yr) else 0,
    'msr_withheld_total': float(yr.get('msr_withhold_this_year', pd.
    ↪Series(dtype=float)).sum()) if len(yr) else np.nan,
    'msr_released_total': float(yr.get('msr_release_this_year', pd.
    ↪Series(dtype=float)).sum()) if len(yr) else np.nan,
    'unsold_frequency': float((yr.get('auction_unsold', pd.
    ↪Series(dtype=float)) > 1e-6).mean()) if len(yr) and 'auction_unsold' in yr.
    ↪columns else np.nan,
    'unsold_total': float(yr.get('auction_unsold', pd.Series(dtype=float)).
    ↪sum() / max(1, yr['episode'].nunique())) if len(yr) and 'auction_unsold' in
    ↪yr.columns and 'episode' in yr.columns else np.nan,
    'bank_total_year12': _safe_mean(yT[bank_cols].sum(axis=1)) if bank_cols
    ↪and not yT.empty else np.nan,
    'bank_total_mean': _safe_mean(yr[bank_cols].sum(axis=1)) if bank_cols
    ↪else np.nan,
    'carry_forward_mean': _safe_mean(yr[cf_cols].sum(axis=1)) if cf_cols
    ↪else np.nan,
    'carry_forward_peak': float(np.nanmax(yr[cf_cols].sum(axis=1))) if
    ↪cf_cols and len(yr) else np.nan,
    'auction_gini': _safe_mean(gini_s),
    'secondary_volume': _safe_mean(ep.get('secondary_volume', pd.
    ↪Series(dtype=float))),
    'secondary_match_rate': _safe_mean(ep.get('secondary_match_rate', pd.
    ↪Series(dtype=float))),
    'secondary_buyers_executed': _safe_mean(yr.
    ↪get('secondary_n_buyers_executed', pd.Series(dtype=float))),

```

```

        'secondary_sellers_executed': _safe_mean(yr.
        ↪get('secondary_n_sellers_executed', pd.Series(dtype=float))),
        'bid_price_agent_var': _safe_mean(yr[bid_cols].var(axis=1)) if bid_cols
        ↪else np.nan,
        'invest_frac_agent_var': _safe_mean(yr[invest_frac_cols].var(axis=1))
        ↪if invest_frac_cols else np.nan,
        'bank_agent_var': _safe_mean(yr[bank_cols].var(axis=1)) if bank_cols
        ↪else np.nan,
        'budget_gate_clip_freq': float((yr[pre_cols].to_numpy() > yr[post_cols].
        ↪to_numpy() + 1e-6).mean()) if pre_cols and post_cols and len(pre_cols) ==
        ↪len(post_cols) and len(yr) else np.nan,
        'quality_score': _safe_mean(ep.get('quality_score', pd.
        ↪Series(dtype=float))),
        'Q_compliance': _safe_mean(ep.get('Q_compliance', pd.
        ↪Series(dtype=float))),
        'Q_volatility': _safe_mean(ep.get('Q_volatility', pd.
        ↪Series(dtype=float))),
        'system_reward': _safe_mean(ep[agent_cols(ep, 'reward')].sum(axis=1))
        ↪if agent_cols(ep, 'reward') else np.nan,
    }
    return row

summary_df = pd.DataFrame([summarize_run(run) for run in runs.values()])
summary_df['variant'] = pd.Categorical(summary_df['variant'],
    ↪categories=VARIANT_ORDER, ordered=True)
summary_df = summary_df.sort_values(['variant', 'seed']).reset_index(drop=True)

agg_cols = [c for c in summary_df.columns if c not in {'variant', 'seed'}]
summary_agg = summary_df.groupby('variant', observed=True)[agg_cols].
    ↪agg(['mean', 'std', 'count'])
print('Per-run converged-window summary:')
display = ipy_display
display(summary_df)
print('Variant-level mean/std/count:')
display(summary_agg)

```

Per-run converged-window summary:

	variant	seed	n_episodes	tail_episodes	mean_price	
↪	final_year_price	within_episode_price_std	floor_touch_freq			
↪	ceiling_touch_freq	green_pct_year12	green_pct_all_years	investment_total		\
0	MAIN	1729	120000	200	66.002251	
↪	80.920555	24.101362	0.432917	0.000000		
↪	73.370737	69.033736	425.494904			
1	MAIN	5041	120000	200	46.675404	
↪	49.446922	5.023928	0.953750	0.000000		
↪	74.735034	69.953752	427.699341			

2	MAIN	6561	120000	200	55.902332	┘
↪	81.258171	19.319714	0.684167	0.000000	┘	
↪	77.008128	70.795429	570.663635			
3	MAIN	8191	120000	200	48.760643	┘
↪	60.960186	6.643657	0.933333	0.000000	┘	
↪	76.416641	70.681649	535.654785			
4	lrf_low_22pct	1729	120000	200	51.216034	┘
↪	58.084503	12.809554	0.777083	0.000000	┘	
↪	73.909456	69.453025	393.140259			
5	lrf_low_22pct	6561	120000	200	45.554134	┘
↪	45.496620	1.783027	0.981250	0.000000	┘	
↪	77.324051	70.914811	568.161560			
6	lrf_low_22pct	8191	120000	200	46.072056	┘
↪	46.973289	3.177407	0.962083	0.000000	┘	
↪	76.453859	70.576549	554.614014			
7	lrf_high_6pct	1729	120000	200	192.776199	┘
↪	249.537537	66.191345	0.049167	0.447083	┘	
↪	73.987585	69.031006	416.916046			
8	lrf_high_6pct	6561	120000	200	148.074753	┘
↪	242.984680	77.353241	0.256250	0.185417	┘	
↪	71.930242	68.406588	283.659851			
9	lrf_high_6pct	8191	120000	200	199.672714	┘
↪	250.000000	64.667984	0.054167	0.503750	┘	
↪	74.712884	69.808352	434.116577			
10	no_market_stability	1729	120000	200	47.725494	┘
↪	60.128422	7.002274	0.907083	0.000000	┘	
↪	73.178118	68.919188	359.090088			
11	no_market_stability	6561	120000	200	45.000000	┘
↪	45.000000	0.000000	1.000000	0.000000	┘	
↪	78.045404	71.131390	617.095337			
12	no_market_stability	8191	120000	200	51.581524	┘
↪	65.872536	12.765271	0.822917	0.000000	┘	
↪	74.303848	69.794327	394.383484			
13	all_financial_objectives	1729	120000	200	54.062515	┘
↪	69.408485	15.348398	0.687083	0.000000	┘	
↪	77.077347	70.496279	580.639893			
14	all_financial_objectives	6561	120000	200	65.305908	┘
↪	89.739021	32.126854	0.637083	0.000000	┘	
↪	69.397491	67.078453	165.694092			
15	all_financial_objectives	8191	120000	200	45.470791	┘
↪	45.961891	1.386763	0.979167	0.000000	┘	
↪	76.025176	70.236808	481.067139			
16	balanced_identical	1729	120000	200	52.406372	┘
↪	64.423126	14.270525	0.797917	0.000000	┘	
↪	61.530280	58.388120	240.191956			

17	balanced_identical	6561	120000	200	92.366455	┐
↪	168.292267	52.823097	0.325417	0.005417		┐
↪	57.720643	56.876922	17.006441			
18	balanced_identical	8191	120000	200	47.754311	┐
↪	52.128658	6.423806	0.936667	0.000000		┐
↪	63.939434	59.873259	355.731049			
19	high_inflation	1729	120000	200	78.897438	┐
↪	156.234283	45.977570	0.535417	0.002917		┐
↪	77.521002	70.630819	693.382996			
compliance_rate penalty_frequency penalty_magnitude_positive ┐						
↪penalty_total tnac_year12 tnac_mean msr_withhold_events msr_release_events┐						
↪ msr_withheld_total msr_released_total unsold_frequency \						
0	0.869062	0.130938		156.334717	1965.	
↪	127344	10.914022	16.967510	1887	0	┐
↪	7971.602539	0.000000	0.012500			
1	0.785885	0.214115		209.079636	4297.	
↪	631875	10.761672	15.657917	1743	0	┐
↪	6572.504883	0.000000	0.236667			
2	0.793229	0.206771		197.966141	3929.	
↪	627500	13.940705	17.986126	1966	0	┐
↪	8731.806641	0.000000	0.006250			
3	0.849896	0.150104		190.844116	2750.	
↪	063750	12.128488	13.808969	1555	4	┐
↪	4541.652344	5.824400	0.617500			
4	0.914271	0.085729		127.469261	1049.	
↪	071953	22.081100	20.637426	1991	0	┐
↪	10154.013672	0.000000	0.097500			
5	0.910990	0.089010		155.307922	1327.	
↪	106094	25.259659	21.772387	1998	0	┐
↪	10682.020508	0.000000	0.020000			
6	0.932656	0.067344		116.879951	755.	
↪	628906	24.359150	21.065989	1993	0	┐
↪	10269.964844	0.000000	0.129167			
7	0.590365	0.409635		226.848587	8920.	
↪	821250	5.100132	13.826380	1500	38	┐
↪	5474.886719	55.331799	0.360000			
8	0.738177	0.261823		169.776062	4267.	
↪	321250	4.441450	13.442517	1365	76	┐
↪	5206.797363	110.663589	0.169167			
9	0.553802	0.446198		244.989502	10494.	
↪	125000	5.135043	14.491602	1604	27	┐
↪	6083.387695	39.314697	0.348750			
10	0.939479	0.060521		303.067139	1760.	
↪	819688	40.081226	30.337761	0	0	┐
↪	0.000000	0.000000	0.002917			

11	0.970208	0.029792	68.187355	195.
↪015820	46.938568	29.779961	0	0
↪	0.000000	0.000000	0.642917	
12	0.893229	0.106771	448.501007	4597.
↪135000	50.723621	34.048069	0	0
↪	0.000000	0.000000	0.040417	
13	0.885469	0.114531	188.570389	2073.
↪331406	12.794334	16.460228	1916	0
↪	7575.656250	0.000000	0.137500	
14	0.817760	0.182240	189.594833	3316.
↪961250	7.334989	14.565124	1546	12
↪	5956.368164	17.473200	0.090833	
15	0.845781	0.154219	403.416840	5972.
↪586250	9.292969	16.582514	1846	4
↪	7752.802734	5.824400	0.021667	
16	0.643125	0.356875	272.679169	9341.
↪988750	8.353298	16.497248	1423	34
↪	6258.333984	58.027802	0.032500	
17	0.450729	0.549271	284.975189	15026.
↪740000	7.186172	14.441931	1279	21
↪	4167.428223	35.840698	0.382083	
18	0.729271	0.270729	355.087585	9228.
↪726250	12.132172	17.678368	1753	1
↪	7315.367188	1.706700	0.101250	
19	0.824844	0.175156	214.555756	3607.
↪754687	14.349155	17.056410	1962	0
↪	8062.132812	0.000000	0.111250	

	unsold_total	bank_total_year12	bank_total_mean	carry_forward_mean	↪
	↪carry_forward_peak	↪auction_gini	↪secondary_volume	↪secondary_match_rate	↪
	↪secondary_buyers_executed	↪secondary_sellers_executed	\		
0	0.121000	9.700365	16.893610	1.022021	↪
↪	5.792100	0.358194	44.094296	0.928835	↪
↪	1.379167		1.586667		
1	4.101403	9.875976	15.598649	2.276619	↪
↪	13.722100	0.315968	30.064665	0.804965	↪
↪	1.045000		1.221250		
2	0.043376	12.684116	18.160870	2.069882	↪
↪	8.656900	0.372063	27.173687	0.733330	↪
↪	0.886667		1.075000		
3	18.466331	11.446197	13.880551	1.427032	↪
↪	25.429499	0.311419	40.524174	0.921735	↪
↪	1.442917		1.818750		
4	1.932069	21.546816	21.550732	0.543291	↪
↪	5.849500	0.376089	64.618172	0.949670	↪
↪	2.195833		2.135833		

5	0.272737	25.152298	22.986145	0.696297	┘
↪	11.392700	0.406414	50.557648	0.950485	┘
↪	1.891250		1.967917		
6	2.127001	24.193804	22.199869	0.398763	┘
↪	5.508700	0.363815	66.013268	0.975915	┘
↪	2.310000		2.325417		
7	12.450872	2.779635	13.175751	4.600039	┘
↪	24.142300	0.437126	33.381817	0.742920	┘
↪	1.091250		1.205833		
8	4.903641	2.615225	12.778189	2.171574	┘
↪	15.573900	0.438944	36.278175	0.880430	┘
↪	1.237500		1.468333		
9	13.301987	2.858296	13.847529	5.446606	┘
↪	23.166199	0.479016	38.733116	0.926740	┘
↪	1.315417		1.463750		
10	0.036561	39.293507	32.729954	0.946582	┘
↪	13.201600	0.400667	29.632587	0.709600	┘
↪	1.104583		1.210833		
11	32.547185	48.637741	32.950836	0.107795	┘
↪	2.991800	0.311798	55.477737	0.938385	┘
↪	2.048750		2.279167		
12	0.665076	51.413944	37.450298	2.382179	┘
↪	22.681200	0.427542	37.561771	0.928825	┘
↪	1.265833		1.247917		
13	2.158987	11.806910	16.561872	1.094150	┘
↪	15.821799	0.331082	35.600517	0.844170	┘
↪	1.247083		1.384167		
14	1.325230	5.546336	14.145050	1.745144	┘
↪	9.013300	0.336059	34.007187	0.812915	┘
↪	1.100833		1.299583		
15	0.306890	7.554500	16.329790	3.098547	┘
↪	20.785801	0.426734	46.497227	0.925105	┘
↪	1.572917		1.788333		
16	0.461700	6.656745	16.017881	4.879100	┘
↪	29.442499	0.395951	38.026695	0.903405	┘
↪	1.125000		1.225417		
17	17.082948	4.586801	13.790069	7.935525	┘
↪	17.721701	0.252882	12.902347	0.442880	┘
↪	0.462917		0.489583		
18	1.336768	10.763366	17.541222	4.703772	┘
↪	27.852402	0.361025	44.639835	0.919280	┘
↪	1.375417		1.455833		
19	1.629637	13.456202	17.295494	1.611099	┘
↪	14.031200	0.442522	40.134010	0.851250	┘
↪	1.312500		1.518750		

	bid_price_agent_var	invest_frac_agent_var	bank_agent_var	quality_score	Q_compliance	Q_volatility	system_reward
0	2343.670166			0.000180		3.937424	0.
↪000000	1.664640	0.869062	0.265321			20.853182	
1	1713.721924			0.000187		3.877197	0.
↪000000	1.002200	0.785886	0.200774			25.154583	
2	2287.109131			0.000176		6.518997	0.
↪000000	1.326430	0.793231	0.230742			12.439602	
3	1349.123047			0.000183		4.999810	0.
↪000000	1.324635	0.849896	0.200732			29.124994	
4	1537.415771			0.000185		5.149585	0.
↪000000	1.616205	0.914272	0.231279			23.367050	
5	1756.679321			0.000184		9.972342	0.
↪000000	1.699400	0.910990	0.123551			30.304459	
6	1542.727295			0.000184		6.832804	0.
↪000000	1.766140	0.932660	0.136225			33.936043	
7	308.748169			0.000175		5.627099	0.
↪000000	-0.972295	0.590362	0.254981			-106.703636	
8	1639.586182			0.000173		2.828022	0.
↪000000	-0.134850	0.738174	0.419338			-52.222401	
9	281.685089			0.000173		7.191718	0.
↪000000	-1.176715	0.553804	0.239957			-101.077789	
10	2153.300781			0.000185		15.528412	0.
↪000000	1.483435	0.939485	0.196201			36.661163	
11	637.880432			0.000181		28.837803	0.
↪000000	1.762015	0.970221	0.176400			53.835018	
12	2193.098145			0.000185		51.015755	0.
↪000000	1.432690	0.893235	0.224592			20.093437	
13	1675.683594			0.000004		3.639301	0.
↪000000	1.578455	0.885466	0.219636			17.739082	
14	2451.641357			0.000015		2.558523	0.
↪000000	1.039070	0.817759	0.382476			6.575173	
15	1530.477661			0.000005		3.729220	0.
↪000000	1.139350	0.845779	0.178858			35.301125	
16	2376.384521			0.000189		6.855068	0.
↪001250	0.362655	0.643120	0.246388			-32.757832	
17	1506.953491			0.000189		7.265933	0.
↪000000	-0.394240	0.450729	0.421462			-64.523438	
18	2126.962158			0.000189		6.263176	0.
↪000625	0.637815	0.729274	0.212717			-22.560694	
19	2849.634277			0.000190		5.579023	0.
↪000000	1.338805	0.824846	0.355136			4.702674	

Variant-level mean/std/count:

			n_episodes		tail_episodes			
mean_price			final_year_price					
within_episode_price_std			floor_touch_freq				\	
			mean	std count	mean	std count		
mean	std count		mean	std count				
mean	std count		mean	std count				
variant								
MAIN			120000.0	0.0	4	200.0	0.0	4 54.
↪335157	8.723993	4	68.146459	15.667479	4			13.
↪772165	9.395298	4	0.751042	0.244947	4			
lrf_low_22pct			120000.0	0.0	3	200.0	0.0	3 47.
↪614075	3.130119	3	50.184804	6.881066	3			5.
↪923329	6.004260	3	0.906806	0.112751	3			
lrf_high_6pct			120000.0	0.0	3	200.0	0.0	3 180.
↪174555	28.012292	3	247.507406	3.923615	3			69.
↪404190	6.926089	3	0.119861	0.118143	3			
no_market_stability			120000.0	0.0	3	200.0	0.0	3 48.
↪102339	3.306905	3	57.000319	10.782136	3			6.
↪589182	6.392654	3	0.910000	0.088578	3			
all_financial_objectives			120000.0	0.0	3	200.0	0.0	3 54.
↪946405	9.947056	3	68.369799	21.907041	3			16.
↪287338	15.391540	3	0.767778	0.184767	3			
balanced_identical			120000.0	0.0	3	200.0	0.0	3 64.
↪175713	24.524455	3	94.948017	63.814753	3			24.
↪505809	24.835345	3	0.686667	0.320451	3			
high_inflation			120000.0	NaN	1	200.0	NaN	1 78.
↪897438	NaN	1	156.234283	NaN	1			45.
↪977570	NaN	1	0.535417	NaN	1			

			ceiling_touch_freq		green_pct_year12			
green_pct_all_years			investment_total					
compliance_rate			penalty_frequency \					
			mean	std count	mean	std count		
std count	mean		mean	std count	mean	std count		
count	mean		std count	mean				
variant								
MAIN			0.000000	0.000000	4	75.382635	1.	
↪651113	4	70.116141	0.812240	4	489.878166	74.460732		
↪4	0.824518	0.041230	4	0.175482				
lrf_low_22pct			0.000000	0.000000	3	75.895788	1.	
↪774386	3	70.314795	0.765239	3	505.305278	97.373649		
↪3	0.919306	0.011678	3	0.080694				
lrf_high_6pct			0.378750	0.169812	3	73.543570	1.	
↪443481	3	69.081982	0.702271	3	378.230825	82.351177		
↪3	0.627448	0.097621	3	0.372552				

no_market_stability			0.000000	0.000000	3	75.175790	2.
↪548103	3	69.948302	1.114110	3	456.856303	139.888588	↪
↪	3	0.934306	0.038749	3	0.065694		
all_financial_objectives			0.000000	0.000000	3	74.166671	4.
↪163601	3	69.270513	1.902808	3	409.133708	216.623679	↪
↪	3	0.849670	0.034021	3	0.150330		
balanced_identical			0.001806	0.003127	3	61.063453	3.
↪135568	3	58.379434	1.498188	3	204.309815	172.189535	↪
↪	3	0.607708	0.142608	3	0.392292		
high_inflation			0.002917	NaN	1	77.521002	↪
↪	NaN	1	70.630819	NaN	1	693.382996	NaN ↪
↪	1	0.824844	NaN	1	0.175156		
				penalty_magnitude_positive			↪
↪	penalty_total			tnac_year12		tnac_mean	↪
↪		msr_withhold_events					
			std count		mean	std	↪
↪count	mean		std count	mean	std count	mean	↪
↪	std count		mean	std count			
variant							
MAIN		0.041230	4		188.556152	22.753878	↪
↪	4	3235.612617	1073.845441	4	11.936222	1.469623	4 ↪
↪1.803164	4		1787.750000	180.553547	4		
lrf_low_22pct		0.011678	3		133.219045	19.848734	↪
↪	3	1043.935651	285.773215	3	23.899970	1.638274	3 ↪
↪0.573120	3		1994.000000	3.605551	3		
lrf_high_6pct		0.097621	3		213.871384	39.250111	↪
↪	3	7894.089167	3237.885933	3	4.892208	0.390759	3 ↪
↪0.530793	3		1489.666667	119.834608	3		
no_market_stability		0.038749	3		273.251834	191.901883	↪
↪	3	2184.323503	2231.407612	3	45.914472	5.394601	3 ↪
↪2.319996	3		0.000000	0.000000	3		
all_financial_objectives		0.034021	3		260.527354	123.746985	↪
↪	3	3787.626302	1991.780931	3	9.807431	2.765794	3 ↪
↪1.131094	3		1769.333333	196.553640	3		
balanced_identical		0.142608	3		304.247314	44.456135	↪
↪	3	11199.151667	3315.272453	3	9.223881	2.585375	3 ↪
↪1.637778	3		1485.000000	243.006173	3		
high_inflation		NaN	1		214.555756	NaN	↪
↪	1	3607.754687	NaN	1	14.349155	NaN	1 ↪
↪	NaN	1	1962.000000	NaN	1		
			msr_release_events		msr_withheld_total		↪
↪		msr_released_total			unsold_frequency		↪
↪	unsold_total			bank_total_year12			

			mean		std count		mean	
↳	std count		mean		std count		mean	↳
↳	std count	mean		std count		mean		↳
variant								
MAIN			1.000000	2.000000	4		6954.391602	↳
↳1840.386780	4		1.456100	2.912200	4		0.218229	0.
↳286948	4	5.683027	8.730334	4	10.926663			
lrf_low_22pct			0.000000	0.000000	3		10368.666341	↳
↳ 277.496475	3		0.000000	0.000000	3		0.082222	0.
↳056164	3	1.443936	1.018960	3	23.630973			
lrf_high_6pct			47.000000	25.709920	3		5588.357259	↳
↳ 449.176274	3		68.436695	37.436210	3		0.292639	0.
↳107078	3	10.218833	4.622721	3	2.751052			
no_market_stability			0.000000	0.000000	3		0.000000	↳
↳ 0.000000	3		0.000000	0.000000	3		0.228750	0.
↳359169	3	11.082941	18.591237	3	46.448397			
all_financial_objectives			5.333333	6.110101	3		7094.942383	↳
↳ 990.004373	3		7.765867	8.896918	3		0.083333	0.
↳058280	3	1.263702	0.927580	3	8.302582			
balanced_identical			18.666667	16.623277	3		5913.709798	↳
↳1602.015700	3		31.858400	28.370947	3		0.171944	0.
↳185204	3	6.293805	9.353910	3	7.335637			
high_inflation			0.000000	NaN	1		8062.132812	↳
↳ NaN	1		0.000000	NaN	1		0.111250	↳
↳NaN	1	1.629637	NaN	1	13.456202			

				bank_total_mean				
↳	carry_forward_mean			carry_forward_peak				↳
↳	auction_gini		secondary_volume					↳
↳	mean	std count		mean	std count		mean	↳
↳	std count	mean		std count				↳
variant								
MAIN			1.410237	4	16.133420	1.830286	4	↳
↳ 1.698889	0.578349	4		13.400150	8.663905	4	0.339411	0.
↳030288	4	35.464205	8.123412	4				
lrf_low_22pct			1.867474	3	22.245582	0.718798	3	↳
↳ 0.546117	0.148787	3		7.583633	3.303147	3	0.382106	0.
↳021928	3	60.396362	8.549082	3				
lrf_high_6pct			0.124031	3	13.267156	0.540498	3	↳
↳ 4.072740	1.699998	3		20.960800	4.690651	3	0.451695	0.
↳023678	3	36.131036	2.678682	3				
no_market_stability			6.349895	3	34.377029	2.663819	3	↳
↳ 1.145519	1.150169	3		12.958200	9.846956	3	0.380002	0.
↳060576	3	40.890699	13.240252	3				

all_financial_objectives	3.196626	3	15.678904	1.333416	3	
↳ 1.979280	1.022505	3	15.206967	5.910284	3	0.364625 0.
↳053845	3	38.701644	6.798015	3		
balanced_identical	3.143749	3	15.783057	1.886569	3	
↳ 5.839466	1.817356	3	25.005534	6.357891	3	0.336619 0.
↳074592	3	31.856292	16.744326	3		
high_inflation		NaN	1	17.295494	NaN	1
↳ 1.611099	NaN	1	14.031200	NaN	1	0.442522
↳ NaN	1	40.134010	NaN	1		

						secondary_match_rate	
↳secondary_buyers_executed						secondary_sellers_executed	
↳ bid_price_agent_var						invest_frac_agent_var \	
						mean	std count
↳ mean		std count				mean	std count
↳ mean		std count				mean	
variant							
MAIN			0.847216	0.094815	4		1.
↳188438	0.266319	4		1.425417	0.339209	4	1923.
↳406067	477.029090	4		0.000182			
lrf_low_22pct			0.958690	0.014923	3		2.
↳132361	0.216470	3		2.143056	0.178859	3	1612.
↳274129	125.086761	3		0.000184			
lrf_high_6pct			0.850030	0.095606	3		1.
↳214722	0.113806	3		1.379306	0.150249	3	743.
↳339813	776.290066	3		0.000174			
no_market_stability			0.858937	0.129418	3		1.
↳473056	0.505043	3		1.579306	0.606381	3	1661.
↳426453	886.640175	3		0.000184			
all_financial_objectives			0.860730	0.057899	3		1.
↳306944	0.241668	3		1.490694	0.261209	3	1885.
↳934204	495.267229	3		0.000008			
balanced_identical			0.755188	0.270583	3		0.
↳987778	0.471473	3		1.056944	0.504675	3	2003.
↳433390	447.685265	3		0.000189			
high_inflation			0.851250	NaN	1		1.
↳312500	NaN	1		1.518750	NaN	1	2849.
↳634277	NaN	1		0.000190			

						bank_agent_var	
↳budget_gate_clip_freq						quality_score	
↳Q_compliance						system_reward \	
						std count	
↳ mean		std count				mean	std count
↳ std count		mean		std count		mean	
variant							

MAIN		4.799274e-06	4	4.833357	1.236396	4	␣
↪	0.000000	0.000000	4	1.329476	0.270479	4	0.824519 ␣
↪	0.041229	4	0.224392	0.030731	4	21.893090	
lrf_low_22pct		8.102691e-07	3	7.318244	2.447751	3	␣
↪	0.000000	0.000000	3	1.693915	0.075118	3	0.919307 ␣
↪	0.011680	3	0.163685	0.058880	3	29.202517	
lrf_high_6pct		1.008109e-06	3	5.215613	2.210758	3	␣
↪	0.000000	0.000000	3	-0.761287	0.552054	3	0.627447 ␣
↪	0.097619	3	0.304758	0.099512	3	-86.667942	
no_market_stability		2.422692e-06	3	31.793990	17.927414	3	␣
↪	0.000000	0.000000	3	1.559380	0.177312	3	0.934314 ␣
↪	0.038753	3	0.199064	0.024223	3	36.863206	
all_financial_objectives		5.792862e-06	3	3.309015	0.651498	3	␣
↪	0.000000	0.000000	3	1.252292	0.286881	3	0.849668 ␣
↪	0.034020	3	0.260323	0.107734	3	19.871793	
balanced_identical		3.223007e-07	3	6.794725	0.504094	3	␣
↪	0.000625	0.000625	3	0.202077	0.534437	3	0.607708 ␣
↪	0.142609	3	0.293522	0.112070	3	-39.947321	
high_inflation		NaN	1	5.579023	NaN	1	␣
↪	0.000000	NaN	1	1.338805	NaN	1	0.824846 ␣
↪	NaN	1	0.355136	NaN	1	4.702674	

	std	count
variant		
MAIN	7.150469	4
lrf_low_22pct	5.369973	3
lrf_high_6pct	29.963044	3
no_market_stability	16.871698	3
all_financial_objectives	14.481243	3
balanced_identical	21.885715	3
high_inflation	NaN	1

1.5 5. Cross-Variant Overview (RQ1/RQ2/RQ3 Bridge)

This section establishes the analysis frame:

- **RQ1 credibility:** whether converged-window quality, compliance, and volatility are stable enough to interpret.
- **RQ2 regulatory sensitivity:** whether LRF/MSR variants shift price, TNAC, compliance, and decarbonization beyond seed noise.
- **RQ3 strategy aggregation:** whether objective/endowment variants alter agent-level strategy dispersion and market outcomes.

```
[11]: def plot_variant_bars(metrics, title, ylabel_map=None, paired_only=False):
      df = summary_df.copy()
      if paired_only:
          df = df[df['seed'].isin(PAIRED_SEEDS)]
```

```

n = len(metrics)
fig, axes = plt.subplots(1, n, figsize=(5.2 * n, 4.5), squeeze=False)
for ax, metric in zip(axes.ravel(), metrics):
    if metric not in df.columns:
        ax.text(0.5, 0.5, f'{metric}\nnot available', ha='center',
        ↪va='center', transform=ax.transAxes)
        continue
    sns.barplot(data=df, x='variant', y=metric, order=[v for v in
    ↪VARIANT_ORDER if v in set(df['variant'].astype(str))],
                errorbar='sd', ax=ax, palette=VARIANT_COLOR)
    sns.stripplot(data=df, x='variant', y=metric, order=[v for v in
    ↪VARIANT_ORDER if v in set(df['variant'].astype(str))],
                  ax=ax, color='black', alpha=0.45, size=3)
    ax.set_title(metric)
    ax.set_xlabel('')
    ax.set_ylabel((ylabel_map or {}).get(metric, metric))
    ax.tick_params(axis='x', rotation=35)
    ax.grid(True, alpha=0.3, axis='y')
    fig.suptitle(title, fontsize=14, y=1.03)
    plt.tight_layout(); plt.show()

plot_variant_bars(['mean_price', 'green_pct_year12', 'compliance_rate',
    ↪'penalty_frequency'],
                  'Converged-window headline outcomes by variant',
                  {'mean_price': '€/tCO ', 'green_pct_year12': 'year-12 green
    ↪%', 'compliance_rate': 'share compliant', 'penalty_frequency': 'agent-year
    ↪share'})
plot_variant_bars(['quality_score', 'within_episode_price_std', 'tnac_year12',
    ↪'bank_total_year12'],
                  'Credibility / stability / banking diagnostics by variant')

```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],

```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

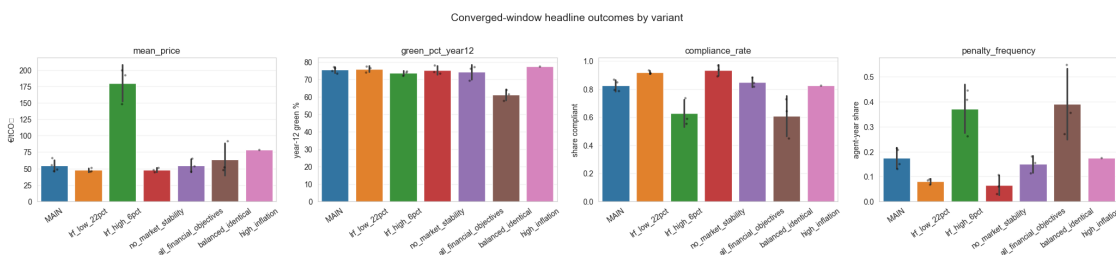
```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:21: UserWarning:
Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.
plt.tight_layout(); plt.show()
C:\Users\danie\AppData\Roaming\Python\Python312\site-
packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT
TWO}) missing from font(s) Arial.
fig.canvas.print_figure(bytes_io, **kw)
```



```
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:
```

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

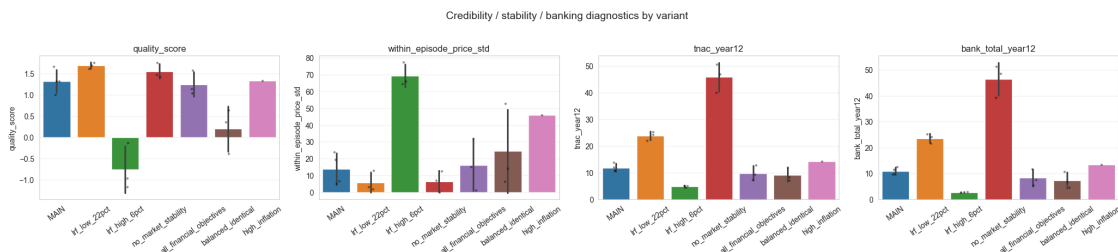
```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
```



```
[12]: variant_matrix = []
_seen_variant_seed = set()
for run in runs.values():
    _key = (run['variant'], run['seed'])
    if _key in _seen_variant_seed:
        continue
    _seen_variant_seed.add(_key)
    flat = _flatten_dict(run['overrides'])
    variant_matrix.append({
```

```

        'variant': run['variant'], 'raw_variant': run['variant_raw'],
        'seed_example': run['seed'],
        'overrides': ', '.join(f'{k}={v}' for k, v in flat.items()) if flat
    else '(base config)',
    })
variant_matrix_df = pd.DataFrame(variant_matrix).drop_duplicates('variant').
    sort_values('variant')
display(variant_matrix_df)

```

	variant	raw_variant	seed_example	
	overrides			
0	MAIN	default	1729	
	(base config)			
13	all_financial_objectives	all_fin	1729	companies.
	reward_weights=[[1.0, 0.0], [1.0, 0...			
16	balanced_identical	balanced	1729	companies.
	initial_mix=[[0.15, 0.35, 0.25, 0.15...			
19	high_inflation	high_inf	1729	penalty.inflation_rate=0.
	04, penalty.inflation...			
7	lrf_high_6pct	lrf_high	1729	
	ets.lrf_phase2=0.063			
4	lrf_low_22pct	lrf_low	1729	
	ets.lrf_phase2=0.022			
10	no_market_stability	no_msr	1729	ets.msr.
	enabled=False, ets.unsold_to_msr=False			

1.6 6. LRF Low (2.2%) vs MAIN

```

[13]: def _yearly_metric(variant, metric_fn, seeds=None):
    rows = []
    for (v, seed), run in runs.items():
        if v != variant or (seeds is not None and seed not in seeds):
            continue
        yr = _tail_yr(run)
        if yr.empty or 'year' not in yr.columns:
            continue
        s = metric_fn(yr, run)
        if s is None or len(s) == 0:
            continue
        df = s.reset_index()
        df.columns = ['year', 'value']
        df['variant'] = variant; df['seed'] = seed
        rows.append(df)
    return pd.concat(rows, ignore_index=True) if rows else pd.
DataFrame(columns=['year', 'value', 'variant', 'seed'])

```

```

def _plot_yearly_variants(variants, metric_fn, title, ylabel, seeds=None,
    ylim=None):
    df = pd.concat([_yearly_metric(v, metric_fn, seeds=seeds) for v in
    variants], ignore_index=True)
    fig, ax = plt.subplots(figsize=(10, 5))
    if df.empty:
        ax.text(0.5, 0.5, 'No year-level data', ha='center', va='center',
    transform=ax.transAxes)
    else:
        for variant in variants:
            sub = df[df['variant'] == variant]
            if sub.empty:
                continue
            by = sub.groupby(['seed', 'year'])['value'].mean().reset_index()
            for seed, g in by.groupby('seed'):
                ax.plot(g['year'], g['value'], color=VARIANT_COLOR.
    get(variant), alpha=0.25, lw=1)
                agg = by.groupby('year')['value'].agg(['mean', 'std']).reset_index()
                ax.plot(agg['year'], agg['mean'], color=VARIANT_COLOR.get(variant),
    lw=2.4, label=f'{variant} mean')
                ax.fill_between(agg['year'], agg['mean'] - agg['std'].fillna(0),
    agg['mean'] + agg['std'].fillna(0),
                                color=VARIANT_COLOR.get(variant), alpha=0.15)
            ax.set(title=title, xlabel='simulation year', ylabel=ylabel)
            if ylim is not None:
                ax.set_ylim(*ylim)
            ax.legend(fontsize=9); ax.grid(True, alpha=0.3)
            plt.tight_layout(); plt.show()
        return df

price_metric = lambda yr, run: yr.groupby('year')['clearing_price'].mean() if
    'clearing_price' in yr.columns else None
green_metric = lambda yr, run: yr.groupby('year')[agent_cols(yr, 'green_frac')].
    mean().mean(axis=1) * 100 if agent_cols(yr, 'green_frac') else None
invest_metric = lambda yr, run: yr.groupby('year')[agent_cols(yr,
    'invest_cost')].sum().sum(axis=1) if agent_cols(yr, 'invest_cost') else None
tnac_metric = lambda yr, run: yr.groupby('year')['tnac'].mean() if 'tnac' in yr.
    columns else None
compliance_metric = lambda yr, run: (yr[agent_cols(yr, 'shortfall')] <= 1e-6).
    mean(axis=1).groupby(yr['year']).mean() if agent_cols(yr, 'shortfall') else
    None

_plot_yearly_variants(['MAIN', 'lrf_low_22pct'], price_metric,
    'LRF low vs MAIN - year-by-year clearing price (seed
    bands)', '€/tCO ', seeds=PAIRED_SEEDS)
_plot_yearly_variants(['MAIN', 'lrf_low_22pct'], green_metric,

```

```

        'LRF low vs MAIN - cumulative/installed green share',
        path, 'green %', seeds=PAIRED_SEEDS, ylim=(0, 100))
    plot_yearly_variants(['MAIN', 'lrf_low_22pct'], invest_metric,
        'LRF low vs MAIN - investment timing by simulation year',
        'system invest cost (M€ per row mean)', seeds=PAIRED_SEEDS)
    plot_yearly_variants(['MAIN', 'lrf_low_22pct'], tnac_metric,
        'LRF low vs MAIN - TNAC trajectory', 'TNAC (Mt)',
        seeds=PAIRED_SEEDS)
    plot_yearly_variants(['MAIN', 'lrf_low_22pct'], compliance_metric,
        'LRF low vs MAIN - compliance rate by year', 'share',
        compliant, seeds=PAIRED_SEEDS, ylim=(0, 1.02))

cols = ['variant', 'seed', 'mean_price', 'floor_touch_freq',
        'green_pct_year12', 'compliance_rate',
        'penalty_frequency', 'penalty_magnitude_positive', 'tnac_year12']
display(summary_df[summary_df['variant'].astype(str).isin(['MAIN',
        'lrf_low_22pct']) & summary_df['seed'].isin(PAIRED_SEEDS)][cols])

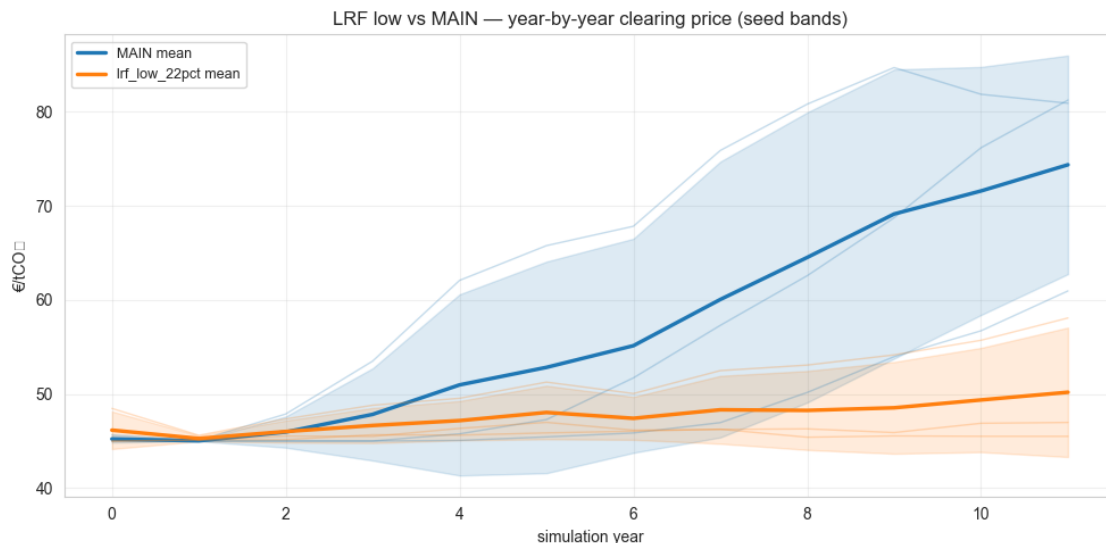
```

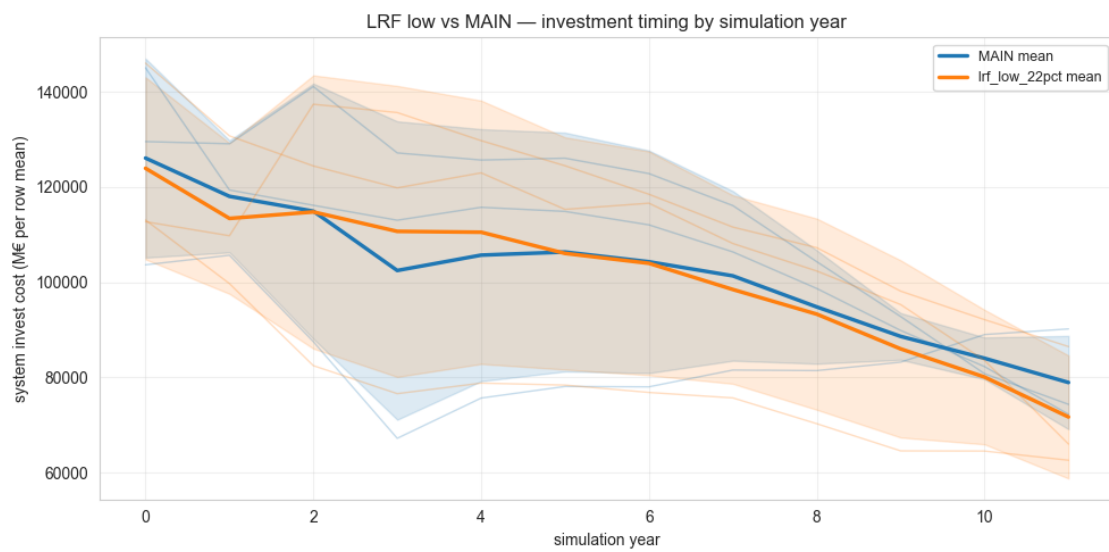
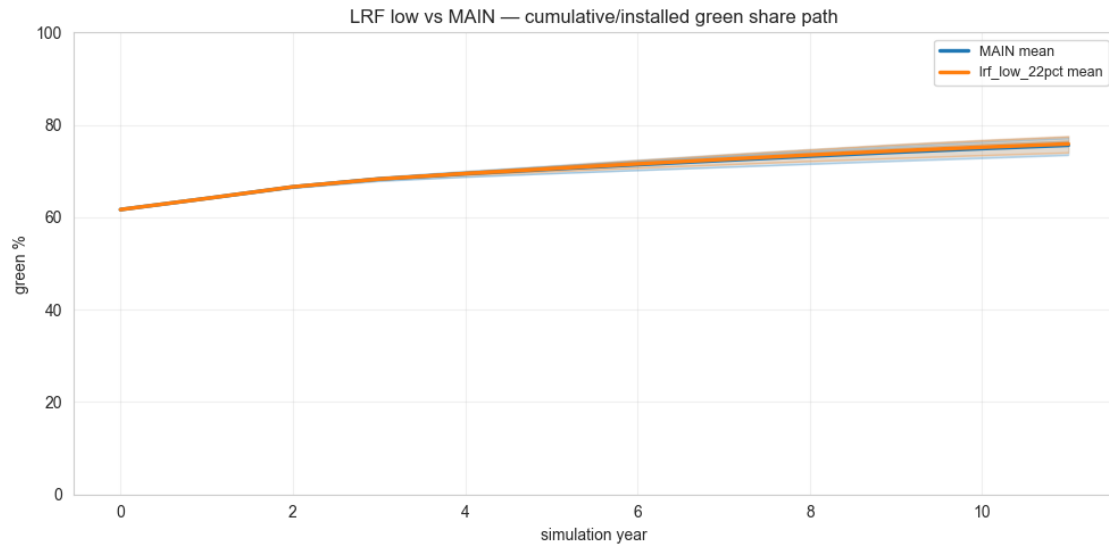
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\2486335180.py:39: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

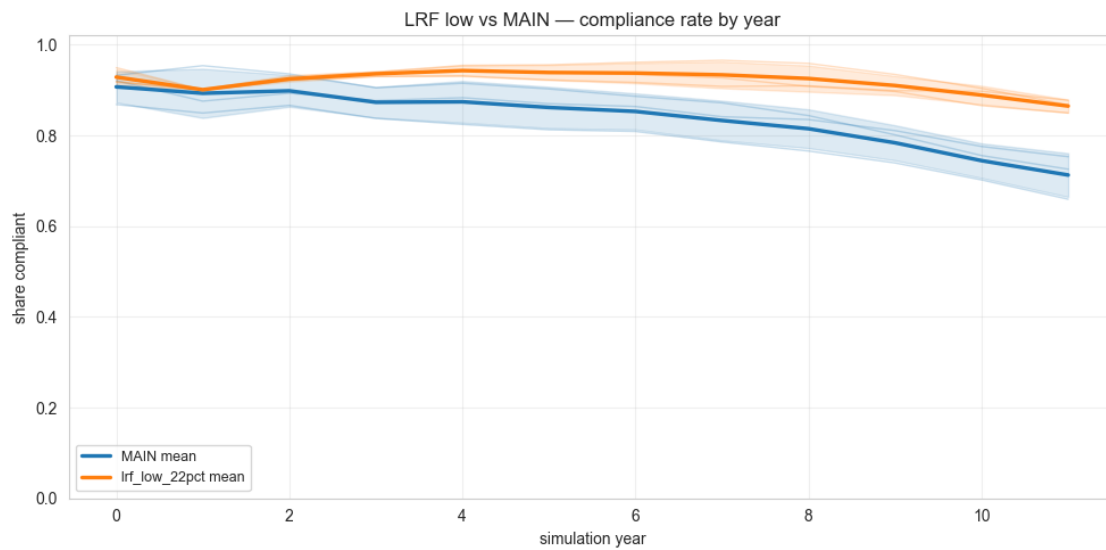
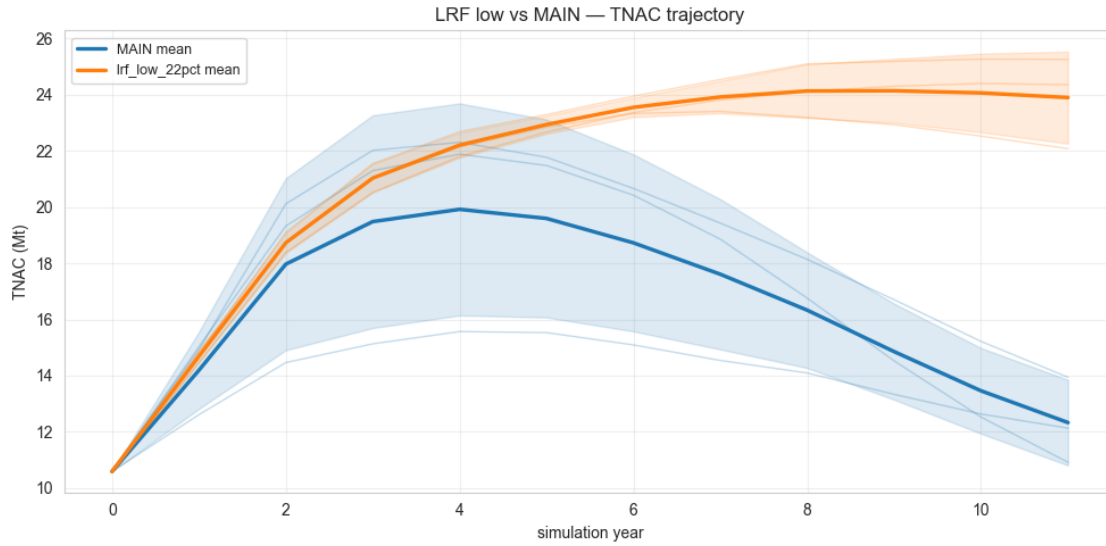
```
plt.tight_layout(); plt.show()
```

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```







	variant	seed	mean_price	floor_touch_freq	green_pct_year12	
	MAIN	1729	66.002251	0.432917	73.370737	0.
↪	compliance_rate		0.130938	156.334717	10.914022	
↪	869062					
	MAIN	6561	55.902332	0.684167	77.008128	0.
↪	793229		0.206771	197.966141	13.940705	
	MAIN	8191	48.760643	0.933333	76.416641	0.
↪	849896		0.150104	190.844116	12.128488	
	lrf_low_22pct	1729	51.216034	0.777083	73.909456	0.
↪	914271		0.085729	127.469261	22.081100	

5	lrf_low_22pct	6561	45.554134	0.981250	77.324051	0.
	↪910990		0.089010	155.307922	25.259659	
6	lrf_low_22pct	8191	46.072056	0.962083	76.453859	0.
	↪932656		0.067344	116.879951	24.359150	

Interpretation prompts for write-up.

- If lrf_low_22pct has high floor_touch_freq and a flat price path close to auction.price_min, the soft cap leaves prices floor-bound.
- Scarcity begins when the LRF-low price path separates upward from the floor / MAIN path and TNAC stops accumulating.
- If green% and investment timing barely move vs MAIN, the lower LRF weakens the regulatory channel without changing strategy.

1.7 7. LRF High (6%) vs MAIN

```
[14]: _plot_yearly_variants(['MAIN', 'lrf_high_6pct'], price_metric,
                          'LRF high vs MAIN - year-by-year clearing price (seed_
                          ↪bands)', '€/tCO ', seeds=PAIRED_SEEDS)
_plot_yearly_variants(['MAIN', 'lrf_high_6pct'], green_metric,
                      'LRF high vs MAIN - green share path', 'green %',
                      ↪seeds=PAIRED_SEEDS, ylim=(0, 100))
_plot_yearly_variants(['MAIN', 'lrf_high_6pct'], invest_metric,
                      'LRF high vs MAIN - investment pull-forward test',
                      ↪'system invest cost (M€ per row mean)', seeds=PAIRED_SEEDS)
_plot_yearly_variants(['MAIN', 'lrf_high_6pct'], compliance_metric,
                      'LRF high vs MAIN - compliance stress by year', 'share_
                      ↪compliant', seeds=PAIRED_SEEDS, ylim=(0, 1.02))

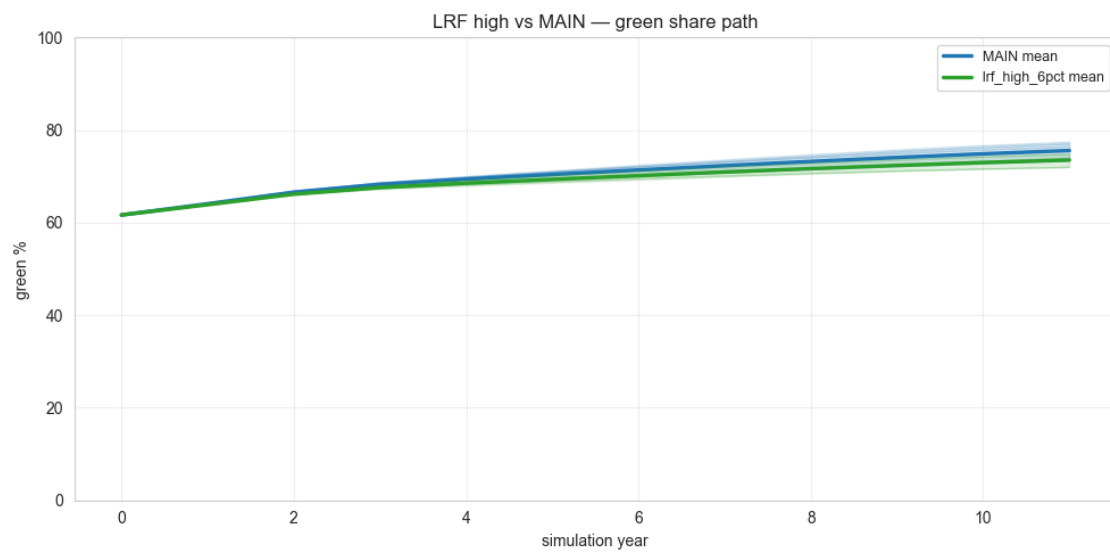
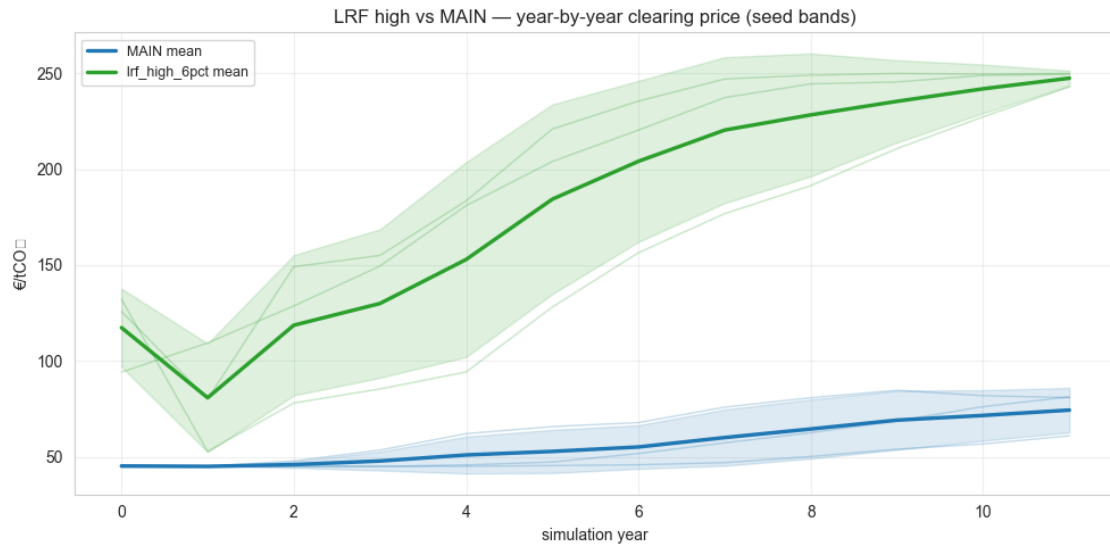
cols = ['variant', 'seed', 'mean_price', 'ceiling_touch_freq',
        ↪'green_pct_year12', 'compliance_rate',
        ↪'carry_forward_mean', 'carry_forward_peak', 'penalty_frequency',
        ↪'penalty_magnitude_positive']
display(summary_df[summary_df['variant'].astype(str).isin(['MAIN',
        ↪'lrf_high_6pct']) & summary_df['seed'].isin(PAIRED_SEEDS)][cols])
```

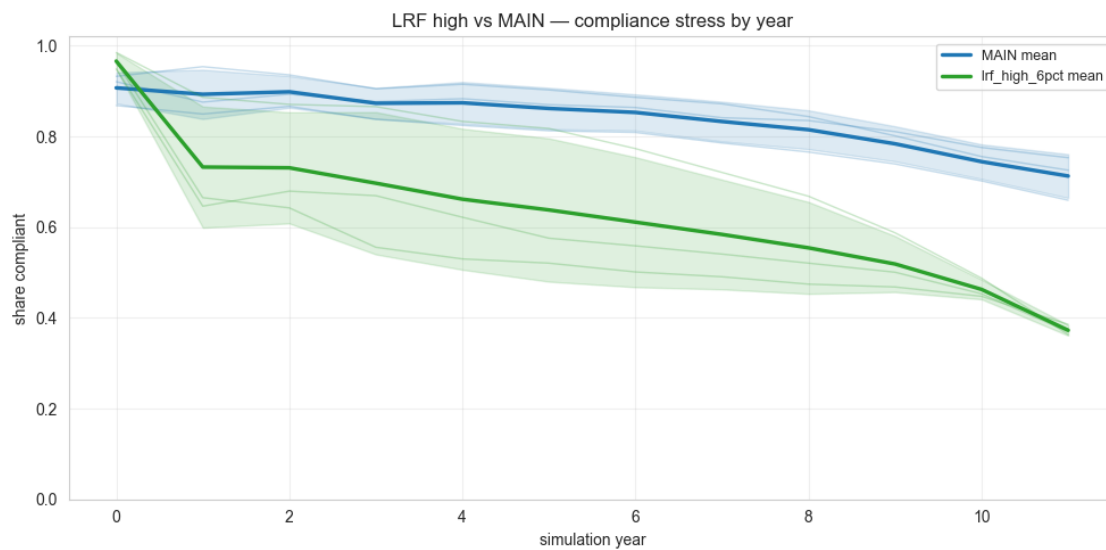
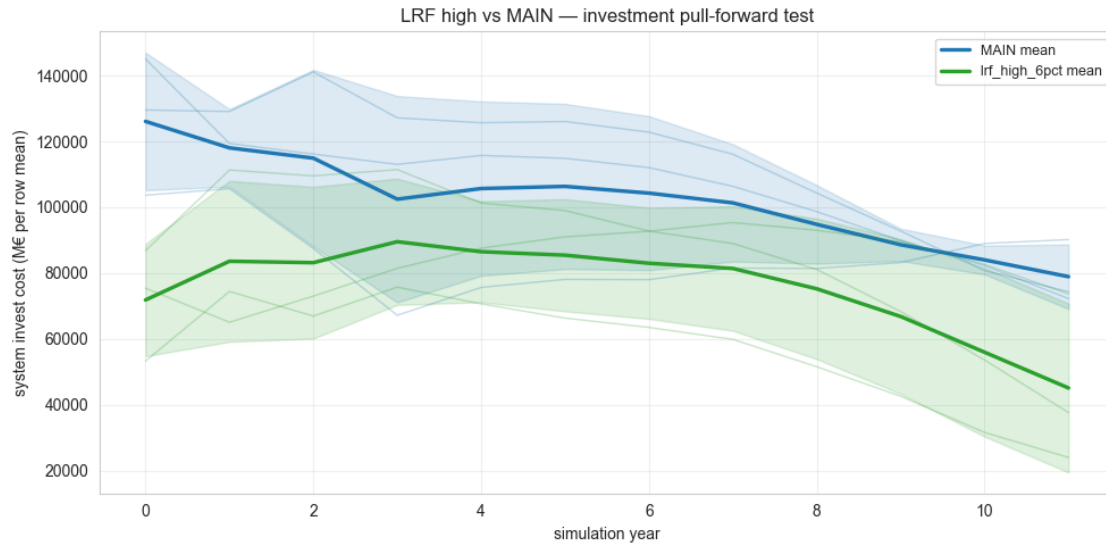
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\2486335180.py:39: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

plt.tight_layout(); plt.show()

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

fig.canvas.print_figure(bytes_io, **kw)





	variant	seed	mean_price	ceiling_touch_freq	green_pct_year12	
0	MAIN	1729	66.002251	0.000000	73.370737	
	↪	0.869062	1.022021	5.792100	0.130938	
	↪	156.334717				
2	MAIN	6561	55.902332	0.000000	77.008128	
	↪	0.793229	2.069882	8.656900	0.206771	
	↪	197.966141				

3	MAIN	8191	48.760643	0.000000	76.416641	␣
	↪0.849896		1.427032	25.429499	0.150104	␣
	↪		190.844116			
7	lrf_high_6pct	1729	192.776199	0.447083	73.987585	␣
	↪0.590365		4.600039	24.142300	0.409635	␣
	↪		226.848587			
8	lrf_high_6pct	6561	148.074753	0.185417	71.930242	␣
	↪0.738177		2.171574	15.573900	0.261823	␣
	↪		169.776062			
9	lrf_high_6pct	8191	199.672714	0.503750	74.712884	␣
	↪0.553802		5.446606	23.166199	0.446198	␣
	↪		244.989502			

Interpretation prompts for write-up.

- If `ceiling_touch_freq` rises materially, the aggressive cap pushes bids toward the price ceiling.
- Scarcity hits earlier if the high-LRF price path rises above MAIN in earlier years, not just in year 12.
- Compliance failure type matters: high mean/peak carry-forward indicates debt cascades, while high positive penalty magnitude indicates direct non-compliance costs.

1.8 8. Cross-LRF Synthesis (2.2% / 4.3% / 6%)

```
[15]: lrf_variants = ['lrf_low_22pct', 'MAIN', 'lrf_high_6pct']
      _plot_yearly_variants(lrf_variants, price_metric,
                           'Cross-LRF price trajectory - 2.2% / MAIN / 6% with seed_␣
                           ↪bands', '€/tCO ', seeds=PAIRED_SEEDS)

lrf_summary = summary_df[summary_df['variant'].astype(str).isin(lrf_variants) &␣
                       ↪summary_df['seed'].isin(PAIRED_SEEDS)].copy()

def _lrf_pp_for_variant(variant):
    vals = []
    for (v, _seed), run in runs.items():
        if v == variant:
            ets = run['config'].get('ets', {})
            val = ets.get('lrf_phase2', ets.get('lrf_phase1'))
            vals.append(float(val) * 100.0 if val is not None else np.nan)
    return float(np.nanmean(vals)) if vals else np.nan

lrf_map = {v: _lrf_pp_for_variant(v) for v in lrf_variants}
lrf_summary['lrf_pp'] = lrf_summary['variant'].astype(str).map(lrf_map)
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
for ax, metric, ylabel in zip(axes, ['green_pct_year12', 'compliance_rate',␣
    ↪'mean_price'], ['year-12 green %', 'compliance %', 'mean price €/t']):
    vals = lrf_summary.copy()
    if metric == 'compliance_rate':
```

```

        vals[metric] *= 100
        sns.barplot(data=vals, x='variant', y=metric, order=lrf_variants,
↳errorbar='sd', palette=VARIANT_COLOR, ax=ax)
        sns.stripplot(data=vals, x='variant', y=metric, order=lrf_variants,
↳color='black', alpha=0.5, size=3, ax=ax)
        ax.set_xlabel(''); ax.set_ylabel(ylabel); ax.tick_params(axis='x',
↳rotation=20); ax.grid(True, alpha=0.3, axis='y')
fig.suptitle('Cross-LRF year-12 / converged outcomes', fontsize=14, y=1.03)
plt.tight_layout(); plt.show()

lrf_means = lrf_summary.groupby('lrf_pp', as_index=False).
↳agg(green_pct=('green_pct_year12', 'mean'), mean_price=('mean_price',
↳'mean'), compliance=('compliance_rate', 'mean'))
lrf_means = lrf_means.sort_values('lrf_pp')
if len(lrf_means) >= 2:
    lrf_means['decarbonization_elasticity_to_next'] = lrf_means['green_pct'].
↳diff() / lrf_means['lrf_pp'].diff()
    overall_elasticity = ((lrf_means['green_pct'].iloc[-1] -
↳lrf_means['green_pct'].iloc[0]) /
                        (lrf_means['lrf_pp'].iloc[-1] - lrf_means['lrf_pp'].
↳iloc[0]))
else:
    overall_elasticity = np.nan
print(f'Overall decarbonization elasticity: {overall_elasticity:.3f}
↳percentage-points green per percentage-point LRF')
display(lrf_means)

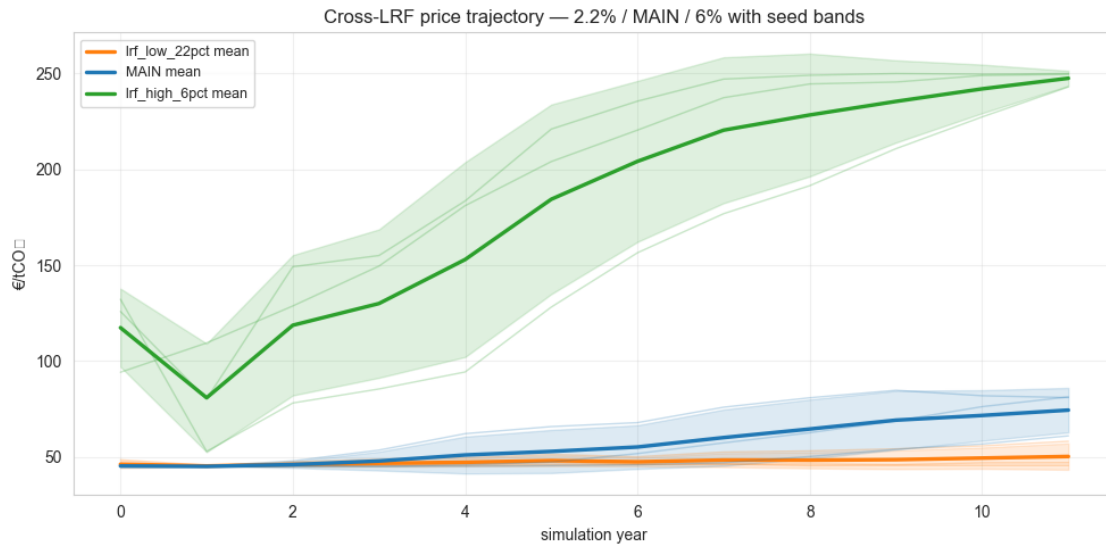
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\2486335180.py:39: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

```
plt.tight_layout(); plt.show()
```

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```



C:\Users\danie\AppData\Local\Temp\ipykernel_28284\561609426.py:23:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=vals, x='variant', y=metric, order=lrf_variants,
errorbar='sd', palette=VARIANT_COLOR, ax=ax)
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\561609426.py:23:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

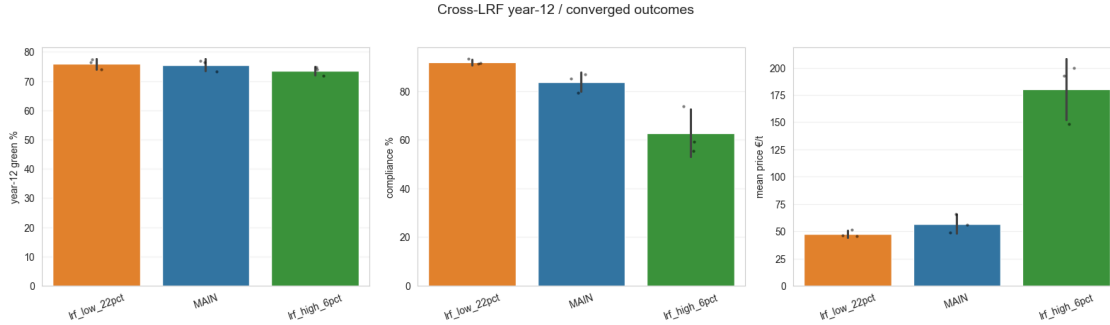
```
sns.barplot(data=vals, x='variant', y=metric, order=lrf_variants,
errorbar='sd', palette=VARIANT_COLOR, ax=ax)
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\561609426.py:23:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=vals, x='variant', y=metric, order=lrf_variants,
errorbar='sd', palette=VARIANT_COLOR, ax=ax)
```



Overall decarbonization elasticity: -0.574 percentage-points green per percentage-point LRF

	lrf_pp	green_pct	mean_price	compliance	decarbonization_elasticity_to_next
0	2.2	75.895788	47.614075	0.919306	NaN
1	4.4	75.598502	56.888409	0.837396	-0.135130
2	6.3	73.543570	180.174555	0.627448	-1.081543

1.9 9. No Market Stability vs MAIN

```
[16]: _plot_yearly_variants(['MAIN', 'no_market_stability'], price_metric,
                        'MSR-off vs MAIN - price path', '€/tCO2',
                        ↪seeds=PAIRED_SEEDS)
_plot_yearly_variants(['MAIN', 'no_market_stability'], tnac_metric,
                        'MSR-off vs MAIN - TNAC surplus trajectory', 'TNAC (Mt)',
                        ↪seeds=PAIRED_SEEDS)

msr_cols = ['variant', 'seed', 'msr_withhold_events', 'msr_release_events',
            ↪'msr_withheld_total', 'msr_released_total',
            'mean_price', 'within_episode_price_std', 'tnac_year12',
            ↪'bank_total_year12', 'unsold_frequency',
            'floor_touch_freq', 'ceiling_touch_freq']
msr_df = summary_df[summary_df['variant'].astype(str).isin(['MAIN',
            ↪'no_market_stability']) & summary_df['seed'].isin(PAIRED_SEEDS)][msr_cols]
display(msr_df)

main_msr_events = summary_df[summary_df['variant'].astype(str).
            ↪eq('MAIN')][['msr_withhold_events', 'msr_release_events']].sum().sum()
if main_msr_events <= 0:
    print('FLAG: MSR never triggered in loaded MAIN runs. MAIN vs
            ↪no_market_stability is a null comparison for MSR mechanics; interpret only
            ↪unsold-to-MSR / config side effects.')
else:
    print(f'MSR triggered in MAIN: total event count across loaded tail windows
            ↪= {int(main_msr_events)}')
```



```

# Across-seed price std by year: the MSR is meant to damp cross-seed noise as
  well as within-episode volatility.
price_paths = pd.concat([yearly_metric(v, price_metric, seeds=PAIRED_SEEDS)
  for v in ['MAIN', 'no_market_stability']], ignore_index=True)
if not price_paths.empty:
    across_seed = price_paths.groupby(['variant', 'year'])['value'].std().
  reset_index(name='across_seed_price_std')
    fig, ax = plt.subplots(figsize=(9, 4.5))
    sns.lineplot(data=across_seed, x='year', y='across_seed_price_std',
  hue='variant', marker='o', ax=ax, palette=VARIANT_COLOR)
    ax.set(title='Price variance across seeds by year', ylabel='std across
  seeds (€/tCO2)', xlabel='year')
    ax.grid(True, alpha=0.3); plt.tight_layout(); plt.show()

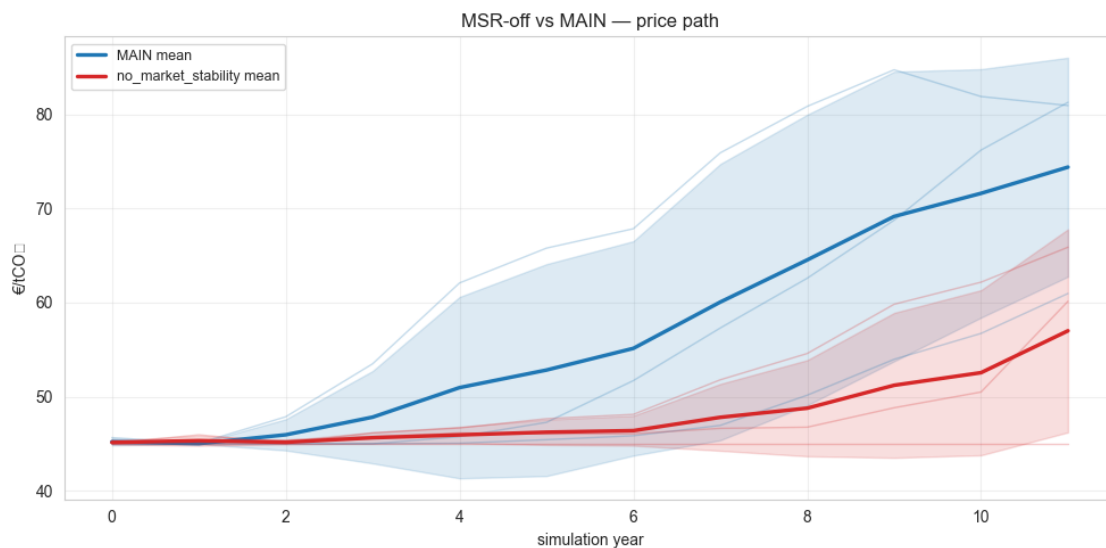
```

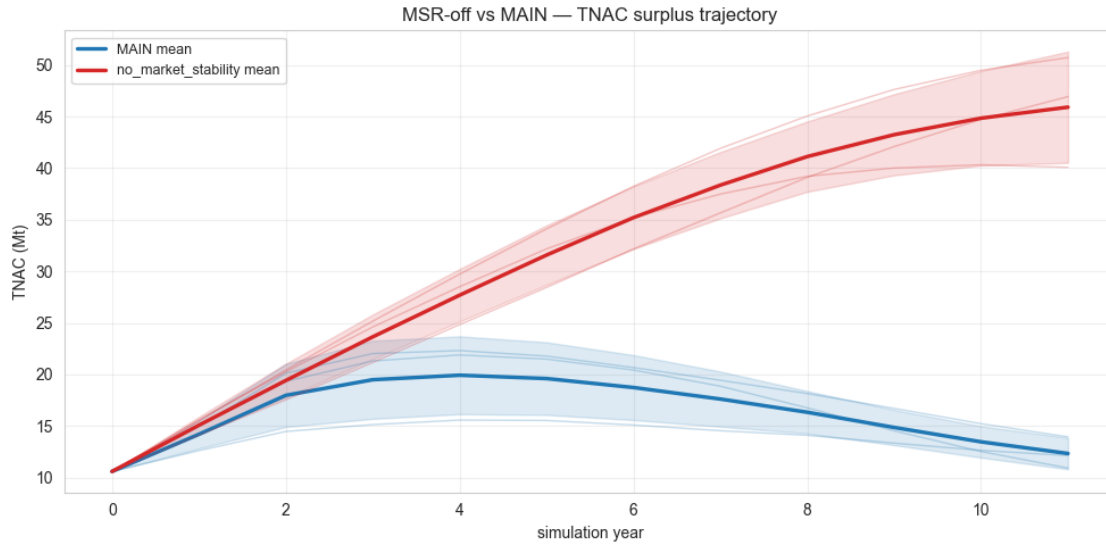
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\2486335180.py:39: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

```
plt.tight_layout(); plt.show()
```

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```





	variant	seed	msr_withhold_events	msr_release_events		
0	MAIN	1729	1887	0		
↪	7971.602539	0.0000	66.002251	24.101362	10.	
↪	914022	9.700365	0.012500	0.432917		
2	MAIN	6561	1966	0		
↪	8731.806641	0.0000	55.902332	19.319714	13.	
↪	940705	12.684116	0.006250	0.684167		
3	MAIN	8191	1555	4		
↪	4541.652344	5.8244	48.760643	6.643657	12.	
↪	128488	11.446197	0.617500	0.933333		
10	no_market_stability	1729	0	0		
↪	0.000000	0.0000	47.725494	7.002274	40.	
↪	081226	39.293507	0.002917	0.907083		
11	no_market_stability	6561	0	0		
↪	0.000000	0.0000	45.000000	0.000000	46.	
↪	938568	48.637741	0.642917	1.000000		
12	no_market_stability	8191	0	0		
↪	0.000000	0.0000	51.581524	12.765271	50.	
↪	723621	51.413944	0.040417	0.822917		
	ceiling_touch_freq					
0	0.0					
2	0.0					
3	0.0					
10	0.0					
11	0.0					

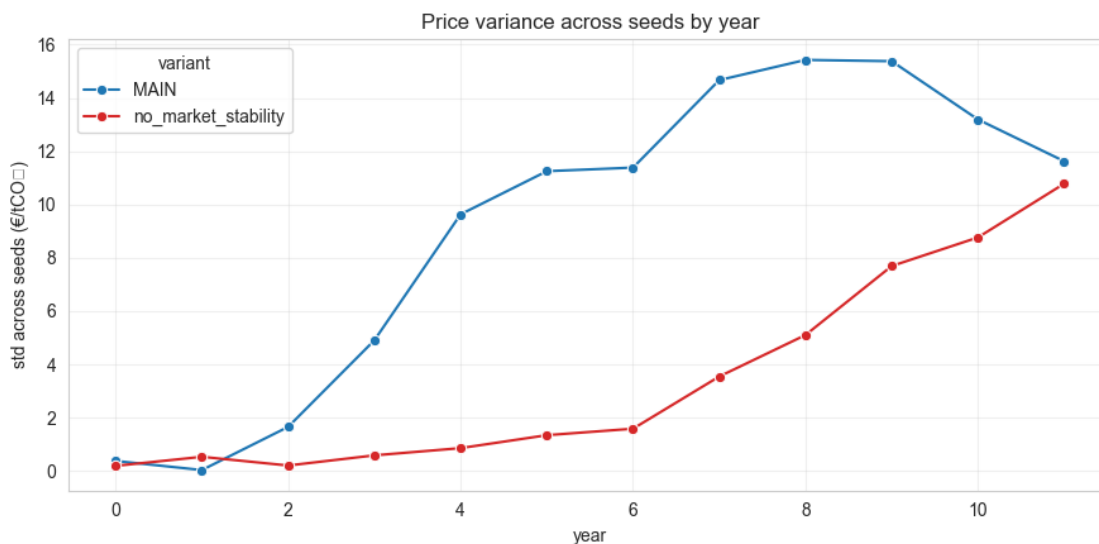
MSR triggered in MAIN: total event count across loaded tail windows = 7155

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\330205796.py:25: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

```
ax.grid(True, alpha=0.3); plt.tight_layout(); plt.show()
```

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```



1.10 10. All-Financial Objectives vs MAIN

```
[17]: _plot_yearly_variants(['MAIN', 'all_financial_objectives'], price_metric,
                             'All-financial vs MAIN - aggregate price level', '€/
                             ↳tCO ', seeds=PAIRED_SEEDS)
_plot_yearly_variants(['MAIN', 'all_financial_objectives'], green_metric,
                             'All-financial vs MAIN - green investment path', 'green_
                             ↳%', seeds=PAIRED_SEEDS, ylim=(0, 100))

cols = ['variant', 'seed', 'mean_price', 'green_pct_year12',
        ↳'investment_total', 'compliance_rate',
        ↳'auction_gini', 'bank_total_year12', 'secondary_volume',
        ↳'secondary_buyers_executed', 'secondary_sellers_executed']
display(summary_df[summary_df['variant'].astype(str).isin(['MAIN',
        ↳'all_financial_objectives']) & summary_df['seed'].isin(PAIRED_SEEDS)][cols])
```

```

# Secondary market directional role by agent group (MAIN has mixed objectives;
↳all_fin should collapse to Financial).
def agent_group_table(variants, seeds=PAIRED_SEEDS):
    rows = []
    for (variant, seed), run in runs.items():
        if variant not in variants or seed not in seeds:
            continue
        yr = _tail_yr(run)
        ep = _tail_ep(run)
        for i in range(N_AGENTS):
            row = {'variant': variant, 'seed': seed, 'agent': f'A{i+1}',
↳'archetype': _agent_archetype(i), 'weighting': _agent_weighting(run, i)}
            for prefix, source in [('bid_price', yr), ('invest_frac_post_clip',
↳yr), ('green_frac', yr), ('bank_end', yr),
                                ('trade_qty', yr), ('sec_qty_action', yr),
↳('shortfall', yr), ('penalty', yr),
                                ('reward', yr), ('reward_base', yr),
↳('reward_shaping', yr), ('invest_cost', yr)]:
                col = f'{prefix}_A{i+1}'
                row[prefix] = _safe_mean(source[col]) if col in source.columns
↳else np.nan
            rows.append(row)
    return pd.DataFrame(rows)

group_df = agent_group_table(['MAIN', 'all_financial_objectives'])
if not group_df.empty:
    display(group_df.groupby(['variant', 'weighting'])[['bid_price',
↳'invest_frac_post_clip', 'green_frac', 'bank_end', 'trade_qty', 'reward',
↳'reward_base', 'reward_shaping']].mean().round(4))

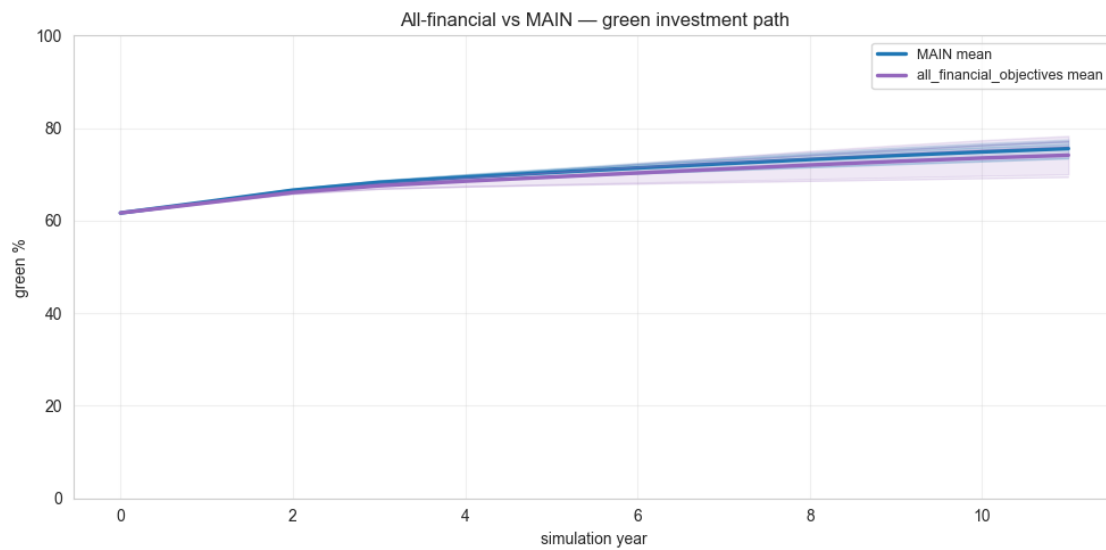
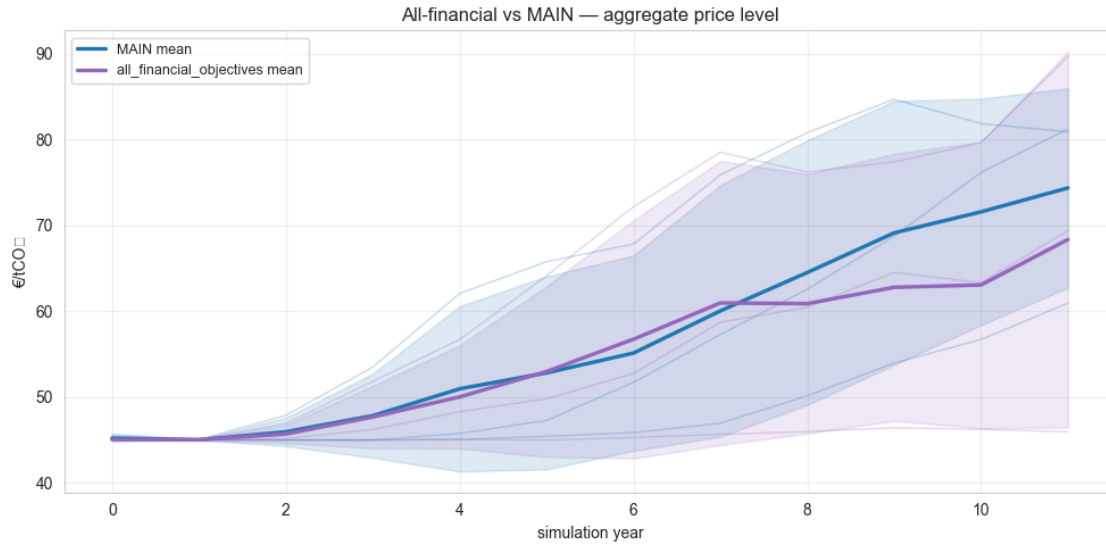
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\2486335180.py:39: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

plt.tight_layout(); plt.show()

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

fig.canvas.print_figure(bytes_io, **kw)



	variant	seed	mean_price	green_pct_year12	
	investment_total	compliance_rate	auction_gini	bank_total_year12	
	secondary_volume	secondary_buyers_executed	secondary_sellers_executed		
0	MAIN	1729	66.002251	73.370737	425.
↳	494904	0.869062	0.358194	9.700365	44.094296
↳		1.379167		1.586667	
2	MAIN	6561	55.902332	77.008128	570.
↳	663635	0.793229	0.372063	12.684116	27.173687
↳		0.886667		1.075000	

3	MAIN	8191	48.760643	76.416641	535.
↪	654785	0.849896	0.311419	11.446197	40.524174
↪		1.442917		1.818750	
13	all_financial_objectives	1729	54.062515	77.077347	580.
↪	639893	0.885469	0.331082	11.806910	35.600517
↪		1.247083		1.384167	
14	all_financial_objectives	6561	65.305908	69.397491	165.
↪	694092	0.817760	0.336059	5.546336	34.007187
↪		1.100833		1.299583	
15	all_financial_objectives	8191	45.470791	76.025176	481.
↪	067139	0.845781	0.426734	7.554500	46.497227
↪		1.572917		1.788333	

				bid_price	invest_frac_post_clip	
↪	green_frac	bank_end	trade_qty	reward	reward_base	reward_shaping
variant			weighting			
MAIN			Balanced ESG	84.7609	0.0303	0.
↪	7244	2.1505	-0.2214 0.4042	0.3609	0.0433	
			Financial	111.0166	0.0059	0.
↪	6790	1.9274	0.2214 0.0293	-0.0123	0.0415	
all_financial_objectives			Financial	102.5544	0.0059	0.
↪	6927	1.9599	-0.0000 0.2070	0.1640	0.0430	

Interpretation prompts for write-up.

- If all-financial green investment remains high, the carbon-cost NPV channel is sufficient to decarbonize even without ESG utility.
- If auction Gini rises, removing ESG may concentrate allowances among agents with stronger financial compliance incentives.
- Secondary-market buy/sell counts and signed `trade_qty` show whether objective removal changes who provides liquidity.

1.11 11. Balanced Identical vs MAIN

```
[18]: cols = ['variant', 'seed', 'bid_price_agent_var', 'invest_frac_agent_var',
↪ 'bank_agent_var', 'mean_price',
        'green_pct_year12', 'compliance_rate', 'secondary_volume']
display(summary_df[summary_df['variant'].astype(str).isin(['MAIN',
↪ 'balanced_identical']) & summary_df['seed'].isin(PAIRED_SEEDS)][cols])

plot_variant_bars(['bid_price_agent_var', 'invest_frac_agent_var',
↪ 'bank_agent_var'],
        'Strategy diversity collapse check: across-agent variance in
↪ converged window', paired_only=True)

balanced_groups = agent_group_table(['balanced_identical'])
if not balanced_groups.empty:
```

```

display(balanced_groups.groupby(['variant', 'weighting'])[['bid_price',
↳ 'invest_frac_post_clip', 'green_frac', 'bank_end', 'trade_qty', 'shortfall',
↳ 'penalty']].agg(['mean', 'std']).round(4))
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, metric in zip(axes, ['bid_price', 'invest_frac_post_clip',
↳ 'bank_end']):
    sns.boxplot(data=balanced_groups, x='weighting', y=metric, ax=ax)
    sns.stripplot(data=balanced_groups, x='weighting', y=metric,
↳ color='black', alpha=0.4, ax=ax)
    ax.set_title(f'balanced_identical: {metric} by objective')
    ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout(); plt.show()

```

	variant	seed	bid_price_agent_var	invest_frac_agent_var	
0	MAIN	1729	2343.670166	0.000180	↳
↳3.937424	66.002251		73.370737	0.869062	44.094296
2	MAIN	6561	2287.109131	0.000176	↳
↳6.518997	55.902332		77.008128	0.793229	27.173687
3	MAIN	8191	1349.123047	0.000183	↳
↳4.999810	48.760643		76.416641	0.849896	40.524174
16	balanced_identical	1729	2376.384521	0.000189	↳
↳6.855068	52.406372		61.530280	0.643125	38.026695
17	balanced_identical	6561	1506.953491	0.000189	↳
↳7.265933	92.366455		57.720643	0.450729	12.902347
18	balanced_identical	8191	2126.962158	0.000189	↳
↳6.263176	47.754311		63.939434	0.729271	44.639835

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],

```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],

```

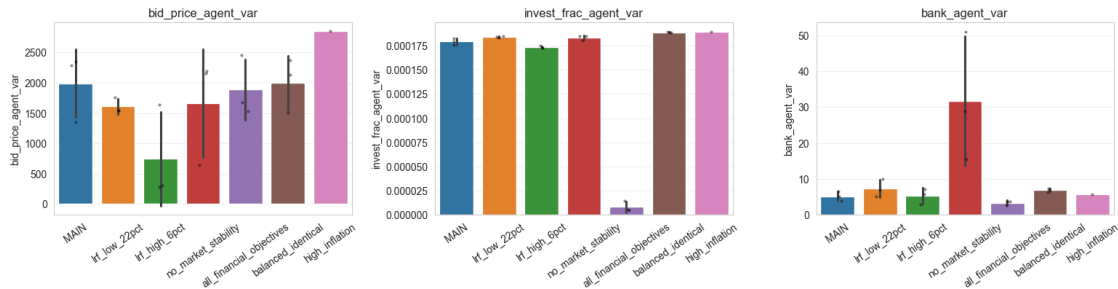
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:

FutureWarning:

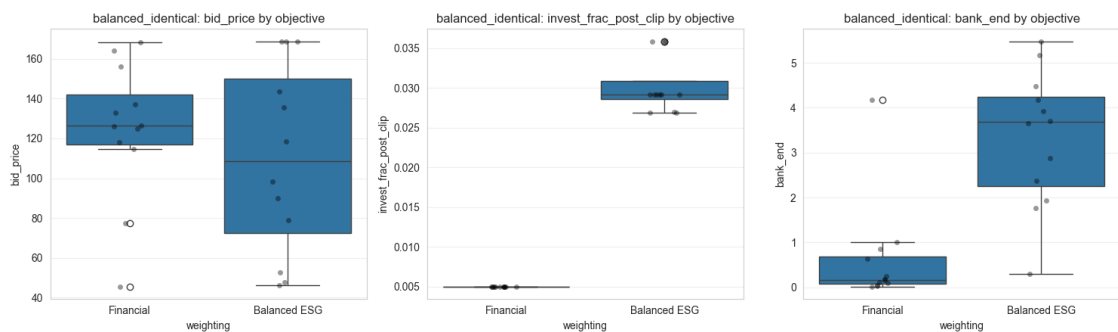
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
```

Strategy diversity collapse check: across-agent variance in converged window



	green_frac		bank_end		bid_price		trade_qty		invest_frac_post_clip		shortfall	
	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std
variant												
balanced_identical	0.5831	0.0284	3.3186	1.5190	-0.2006	1.0169	0.2678	0.4118	43.9083	67.7949	0.0303	0.0035
Financial	0.5845	0.0297	0.6271	1.1654	0.2006	0.5486	1.1921	0.7122	189.4074	114.6827	0.0050	0.0000



1.12 12. Within-MAIN Agent Decomposition (Sub-RQ 3)

```
[20]: main_seeds = sorted({seed for (variant, seed) in runs if variant == 'MAIN'})
main_agents = agent_group_table(['MAIN'], seeds=main_seeds)
if main_agents.empty:
    print('No MAIN agent-level data available.')
else:
    metrics = ['bid_price', 'invest_frac_post_clip', 'green_frac', 'bank_end',
    ↪ 'trade_qty', 'sec_qty_action',
    ↪ 'shortfall', 'penalty', 'reward', 'reward_base',
    ↪ 'reward_shaping', 'invest_cost']
    display(main_agents.groupby(['archetype', 'weighting'])[metrics].mean().
    ↪ round(4))
    display(main_agents.groupby('weighting')[metrics].mean().round(4))

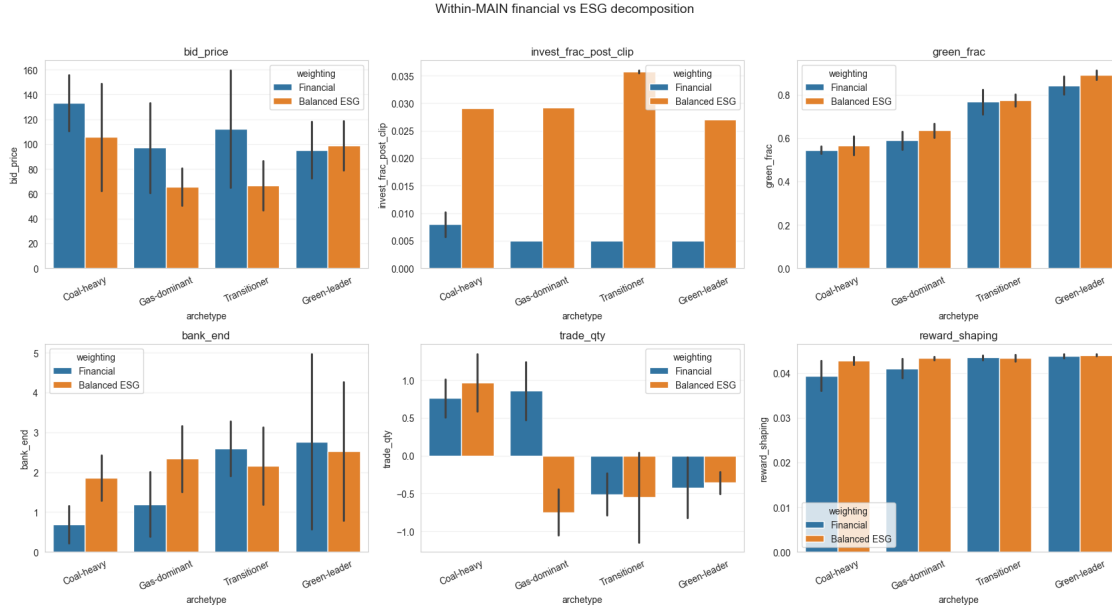
    fig, axes = plt.subplots(2, 3, figsize=(17, 9))
    for ax, metric in zip(axes.ravel(), ['bid_price', 'invest_frac_post_clip',
    ↪ 'green_frac', 'bank_end', 'trade_qty', 'reward_shaping']):
        sns.barplot(data=main_agents, x='archetype', y=metric, hue='weighting',
    ↪ errorbar='sd', ax=ax)
        ax.set_title(metric); ax.tick_params(axis='x', rotation=25); ax.
    ↪ grid(True, alpha=0.3, axis='y')
        plt.suptitle('Within-MAIN financial vs ESG decomposition', fontsize=14, y=1.
    ↪ 02)
        plt.tight_layout(); plt.show()

    # Secondary-market direction: positive trade_qty = buyer, negative = seller.
    sm = main_agents.copy()
    sm['secondary_role'] = np.select([sm['trade_qty'] > 1e-6, sm['trade_qty'] <
    ↪ -1e-6], ['net buyer', 'net seller'], default='neutral')
    display(pd.crosstab([sm['archetype'], sm['weighting']],
    ↪ sm['secondary_role'], normalize='index').round(3))

    # Reward composition proxy: direct ESG component is not logged separately,
    ↪ so reward_shaping + ESG diagnostics are used as evidence.
    esg_cols = [c for c in ['reward', 'reward_base', 'reward_shaping',
    ↪ 'penalty', 'invest_cost'] if c in main_agents.columns]
    print('Reward composition proxy (reward_base is financial+ESG-penalty; ESG
    ↪ term is not separately logged in CSV):')
    display(main_agents.groupby('weighting')[esg_cols].mean().round(4))

    bid_price  invest_frac_post_clip  green_frac
    ↪ bank_end  trade_qty  sec_qty_action  shortfall  penalty  reward  reward_base
    ↪ reward_shaping  invest_cost
    archetype    weighting
```

Coal-heavy	Balanced ESG	105.6322			0.0291	0.5660	1.
↪8552	0.9698	2.7758	0.1281	19.9979	-0.1615	-0.2044	⌋
↪	0.0428	78.8037					
	Financial	133.2186			0.0080	0.5451	0.
↪6912	0.7628	2.9439	0.8226	130.0833	-0.6429	-0.6823	⌋
↪	0.0394	56.7839					
Gas-dominant	Balanced ESG	65.6467			0.0292	0.6349	2.
↪3388	-0.7483	0.8249	0.0908	15.0288	0.4313	0.3880	⌋
↪	0.0433	88.5554					
	Financial	97.0259			0.0050	0.5888	1.
↪1946	0.8579	2.8191	0.4582	71.6212	-0.8407	-0.8818	⌋
↪	0.0411	32.5865					
Green-leader	Balanced ESG	98.6550			0.0270	0.8911	2.
↪5323	-0.3581	-0.4313	0.0079	1.3283	0.8730	0.8289	⌋
↪	0.0441	74.4609					
	Financial	95.2722			0.0050	0.8427	2.
↪7665	-0.4237	-2.1281	0.0259	4.2336	0.8971	0.8533	⌋
↪	0.0438	25.2889					
Transitioner	Balanced ESG	66.6146			0.0357	0.7737	2.
↪1573	-0.5508	-1.6699	0.1102	18.1832	0.4259	0.3825	⌋
↪	0.0434	64.5023					
	Financial	112.1309			0.0050	0.7671	2.
↪5976	-0.5096	-1.4323	0.0551	9.1581	0.8422	0.7987	⌋
↪	0.0435	68.8967					
	bid_price	invest_frac	post_clip	green_frac	bank_end	trade_qty	⌋
↪sec_qty_action	shortfall	penalty	reward	reward_base	reward_shaping		⌋
↪invest_cost							
weighting							
Balanced ESG	84.1371		0.0303	0.7164	2.2209	-0.1718	⌋
↪	0.3749	0.0843	13.6346	0.3922	0.3488	0.0434	⌋
↪76.5806							
Financial	109.4119		0.0057	0.6859	1.8125	0.1718	⌋
↪	0.5506	0.3405	53.7740	0.0639	0.0220	0.0420	⌋
↪45.8890							



secondary_role		net buyer	net seller
archetype	weighting		
Coal-heavy	Balanced ESG	1.0	0.0
	Financial	1.0	0.0
Gas-dominant	Balanced ESG	0.0	1.0
	Financial	1.0	0.0
Green-leader	Balanced ESG	0.0	1.0
	Financial	0.0	1.0
Transitioner	Balanced ESG	0.0	1.0
	Financial	0.0	1.0

Reward composition proxy (reward_base is financial+ESG-penalty; ESG term is not separately logged in CSV):

	reward	reward_base	reward_shaping	penalty	invest_cost
weighting					
Balanced ESG	0.3922	0.3488	0.0434	13.6346	76.5806
Financial	0.0639	0.0220	0.0420	53.7740	45.8890

Reward decomposition caveat. The CSV exposes reward, reward_base, reward_shaping, penalties, and investment costs, but not a clean year-level financial_term vs esg_term split. Use the weighting comparison plus $\text{esg_vs_penalty_ratio_A}^* / \text{compliance_gate_A}^*$ from episode logs as the diagnostic evidence, and state that the exact ESG component is inferred rather than directly observed.

1.12.1 12a. MAIN Reward Composition Proxy — Cost / ESG / Penalty

The simulator does not currently log the final esg_signal and full financial cost norm as separate CSV columns. This cell reconstructs the closest auditable proxy from year-level cost buckets and

the documented ESG flow formula:

- **Cost proxy:** auction + secondary + MAC + investment + collateral costs.
- **Penalty:** logged penalty cost.
- **ESG flow proxy:** $(\text{initial_ef} - \text{realised_ef}) / \text{initial_ef} \times (\text{real_anchor_t} / \text{real_anchor_0})$, gated by observed compliance where possible.

Use this to verify whether ESG-weighted agents are exposed to a materially larger ESG-flow channel than financial agents, while noting the residual caveat in the thesis text.

```
[21]: def _initial_ef_for_agent(run, i):
    tech = run['config'].get('technologies', {})
    efs = np.asarray(tech.get('emission_factors', []), dtype=float)
    mix = run['config'].get('companies', {}).get('initial_mix', [])
    if i >= len(mix) or len(efs) == 0:
        return np.nan
    m = np.asarray(mix[i], dtype=float)
    n = min(len(m), len(efs))
    return float(np.dot(m[:n], efs[:n]))

def _agent_reward_proxy(variants, seeds=None):
    """Reconstruct auditable reward-channel proxies from logged CSV fields.

    The true ESG signal is not directly logged separately; this function must
    remain labelled as an inferred proxy in any downstream reporting. The
    ESG-flow proxy mirrors the documented formula's flow component: avoided_
    ↪ emission intensity
    relative to the agent's initial mix, multiplied by the real anchor ratio
    and a binary observed-compliance gate. It is therefore a directional
    diagnostic, not a replacement for the simulator's internal reward term.
    """
    rows = []
    for (variant, seed), run in runs.items():
        if variant not in variants or (seeds is not None and seed not in seeds):
            continue
        yr = _tail_yr(run)
        if yr.empty:
            continue
        output_twh = float(run['config'].get('companies', {}).get('output_twh', 0
        ↪ 10.0))
        if 'fundamental_anchor' in yr.columns and 'inflation_factor' in yr.
        ↪ columns:
            infl = yr['inflation_factor'].replace(0, np.nan)
            anchor_real = yr['fundamental_anchor'] / infl
            anchor0 = anchor_real[yr['year'].eq(0)].dropna()
            # Tail windows contain many episodes; use the median year-0 real
            # anchor rather than silently selecting the first episode.
            anchor0 = float(anchor0.median()) if len(anchor0) else np.nan
```

```

        anchor_ratio = anchor_real / max(anchor0, 1.0)
    else:
        anchor_ratio = pd.Series(1.0, index=yr.index)
    for i in range(N_AGENTS):
        init_ef = _initial_ef_for_agent(run, i)
        emissions_col = f'emissions_A{i+1}'
        short_col = f'shortfall_A{i+1}'
        if emissions_col in yr.columns and np.isfinite(init_ef) and init_ef_
↪ > 0:
            realised_ef = yr[emissions_col] / max(output_twh, EPS)
            ef_ratio = ((init_ef - realised_ef) / init_ef).clip(lower=0)
        else:
            ef_ratio = pd.Series(np.nan, index=yr.index)
        if short_col in yr.columns:
            compliance_gate_proxy = (yr[short_col] <= 1e-6).astype(float)
        else:
            compliance_gate_proxy = 1.0
        # Proxy only: direct esg_signal/esg_flow_term is not persisted in
        # year_log, so this reconstructs direction/magnitude from available
        # emissions, anchor, and compliance-gate observables.
        esg_flow_proxy = ef_ratio * anchor_ratio * compliance_gate_proxy
        cost_parts = []
        for prefix in ['auction_cost', 'secondary_net', 'mac_cost',
↪ 'invest_cost', 'collateral_cost']:
            col = f'{prefix}_A{i+1}'
            if col in yr.columns:
                cost_parts.append(yr[col].abs() if prefix ==
↪ 'secondary_net' else yr[col])
            cost_proxy = sum(cost_parts) if cost_parts else pd.Series(np.nan,
↪ index=yr.index)
            pen_col = f'penalty_A{i+1}'
            rows.append({
                'variant': variant, 'seed': seed, 'agent': f'A{i+1}',
↪ 'archetype': _agent_archetype(i),
                'weighting': _agent_weighting(run, i),
                'cost_proxy': _safe_mean(cost_proxy),
                'penalty': _safe_mean(yr[pen_col]) if pen_col in yr.columns
↪ else np.nan,
                'esg_flow_proxy': _safe_mean(esg_flow_proxy),
                'reward_base': _safe_mean(yr[f'reward_base_A{i+1}']) if
↪ f'reward_base_A{i+1}' in yr.columns else np.nan,
                'reward_shaping': _safe_mean(yr[f'reward_shaping_A{i+1}']) if
↪ f'reward_shaping_A{i+1}' in yr.columns else np.nan,
            })
    return pd.DataFrame(rows)

```

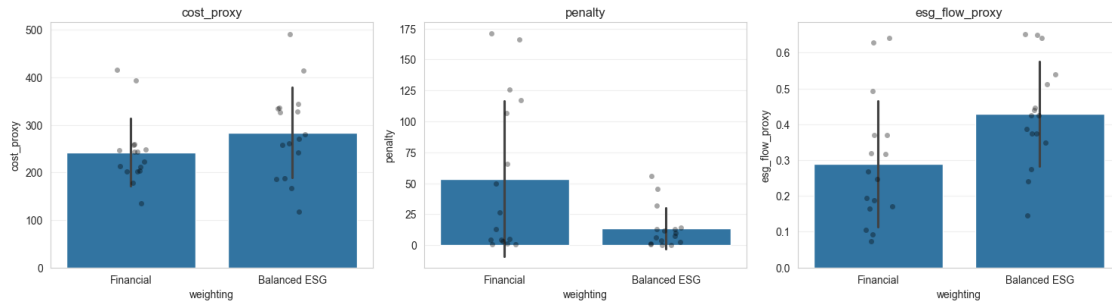
```

main_reward_proxy = _agent_reward_proxy(['MAIN'])
if main_reward_proxy.empty:
    print('No MAIN reward-proxy data available.')
else:
    display(main_reward_proxy.groupby(['archetype',
    ↪'weighting'])[['cost_proxy', 'penalty', 'esg_flow_proxy', 'reward_base',
    ↪'reward_shaping']].mean().round(4))
    display(main_reward_proxy.groupby('weighting')[['cost_proxy', 'penalty',
    ↪'esg_flow_proxy', 'reward_base', 'reward_shaping']].mean().round(4))
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
    for ax, metric in zip(axes, ['cost_proxy', 'penalty', 'esg_flow_proxy']):
        sns.barplot(data=main_reward_proxy, x='weighting', y=metric,
    ↪errorbar='sd', ax=ax)
        sns.stripplot(data=main_reward_proxy, x='weighting', y=metric,
    ↪color='black', alpha=0.35, ax=ax)
        ax.set_title(metric); ax.grid(True, alpha=0.3, axis='y')
    plt.suptitle('MAIN reward composition proxy: financial cost / penalty /
    ↪ESG-flow channel', fontsize=14, y=1.03)
    plt.tight_layout(); plt.show()

```

		cost_proxy	penalty	esg_flow_proxy	reward_base	
	↪reward_shaping					
archetype	weighting					
Coal-heavy	Balanced ESG	341.1468	19.9979	0.2918	-0.2044	↪
	↪0.0428					
	Financial	231.8318	130.0833	0.1316	-0.6823	↪
	↪0.0394					
Gas-dominant	Balanced ESG	358.0943	15.0288	0.3691	0.3880	↪
	↪0.0433					
	Financial	272.3957	71.6212	0.1780	-0.8818	↪
	↪0.0411					
Green-leader	Balanced ESG	199.8938	1.3283	0.6137	0.8289	↪
	↪0.0441					
	Financial	195.9306	4.2336	0.4218	0.8533	↪
	↪0.0438					
Transitioner	Balanced ESG	236.7987	18.1832	0.4436	0.3825	↪
	↪0.0434					
	Financial	268.9285	9.1581	0.4293	0.7987	↪
	↪0.0435					
		cost_proxy	penalty	esg_flow_proxy	reward_base	reward_shaping
weighting						
Balanced ESG		283.9834	13.6346	0.4295	0.3488	0.0434
Financial		242.2716	53.7740	0.2901	0.0220	0.0420

MAIN reward composition proxy: financial cost / penalty / ESG-flow channel



1.13 13. High-Inflation Single-Seed Probe

```
[23]: # Colorblind-safe palette (Okabe-Ito) + style redundancy (linestyle)
VARIANT_COLOR.update({
    'MAIN': '#0072B2',          # blue
    'high_inflation': '#D55E00', # vermillion
})
variant_style = {
    'MAIN': '-',
    'high_inflation': '--',
}

_plot_yearly_variants(
    ['MAIN', 'high_inflation'],
    price_metric,
    'High-inflation probe vs MAIN(seed 1729) - price path',
    '€/tCO ',
    seeds=[1729],
)

_plot_yearly_variants(
    ['MAIN', 'high_inflation'],
    compliance_metric,
    'High-inflation probe vs MAIN(seed 1729) - compliance path',
    'share compliant',
    seeds=[1729],
    ylim=(0, 1.02),
)

hi_cols = ['variant', 'seed', 'mean_price', 'green_pct_year12',
    ↪ 'compliance_rate', 'penalty_frequency',
    'penalty_magnitude_positive', 'budget_gate_clip_freq',
    ↪ 'quality_score', 'Q_volatility',
    'within_episode_price_std', 'system_reward']
hi_df = summary_df[
```

```

summary_df['variant'].astype(str).isin(['MAIN', 'high_inflation']) &
summary_df['seed'].eq(1729)
][hi_cols]
display(hi_df)

# Episode-level diagnostics for the single seed.
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
for ax, metric in zip(axes, ['system_reward', 'quality_score', 'entropy_coef']):
    for variant in ['MAIN', 'high_inflation']:
        run = runs.get((variant, 1729))
        if run is None or run['ep_df'].empty:
            continue
        ep = run['ep_df']
        if metric == 'system_reward':
            cols = agent_cols(ep, 'reward')
            s = ep[cols].sum(axis=1) if cols else pd.Series(dtype=float)
        elif metric in ep.columns:
            s = ep[metric]
        else:
            continue
        x = ep['episode'] if 'episode' in ep.columns else np.arange(len(ep))
        ax.plot(
            x,
            s.rolling(ROLL, min_periods=1).mean(),
            label=variant,
            color=VARIANT_COLOR.get(variant),
            linestyle=variant_style.get(variant, '-'),
            linewidth=2.2
        )
        ax.set_title(metric)
        ax.grid(True, alpha=0.3)
        ax.legend(fontsize=8)
plt.suptitle('High-inflation convergence/pathology probe', fontsize=14, y=1.03)
plt.tight_layout()
plt.show()

if not hi_df.empty and {'compliance_rate', 'quality_score',
    'budget_gate_clip_freq'}.issubset(hi_df.columns):
    main = hi_df[hi_df['variant'].astype(str) == 'MAIN']
    high = hi_df[hi_df['variant'].astype(str) == 'high_inflation']
    if len(main) and len(high):
        comp_drop = float(main['compliance_rate'].iloc[0] -
    high['compliance_rate'].iloc[0])
        q_drop = float(main['quality_score'].iloc[0] - high['quality_score'].
    iloc[0])
        gate_jump = (

```



```

        float(high['budget_gate_clip_freq'].iloc[0] -
↪main['budget_gate_clip_freq'].iloc[0])
        if pd.notna(high['budget_gate_clip_freq'].iloc[0]) and pd.
↪notna(main['budget_gate_clip_freq'].iloc[0])
            else np.nan
    )
    promote = (comp_drop > 0.05) or (q_drop > 0.10) or (pd.notna(gate_jump)
↪and gate_jump > 0.05)
    print('Decision:', 'YES - warrants a full 3-seed run.' if promote else
↪'NO - single-seed probe does not show a large qualitative break.')
    print(f'Justification: compliance drop={comp_drop:.3f}, quality
↪drop={q_drop:.3f}, budget-gate clip jump={gate_jump:.3f}.')

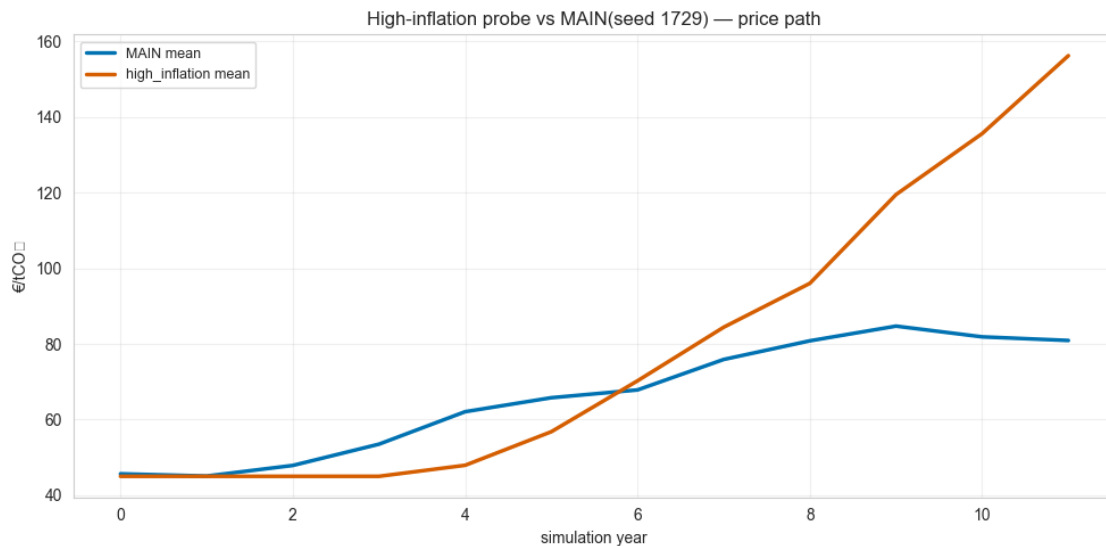
```

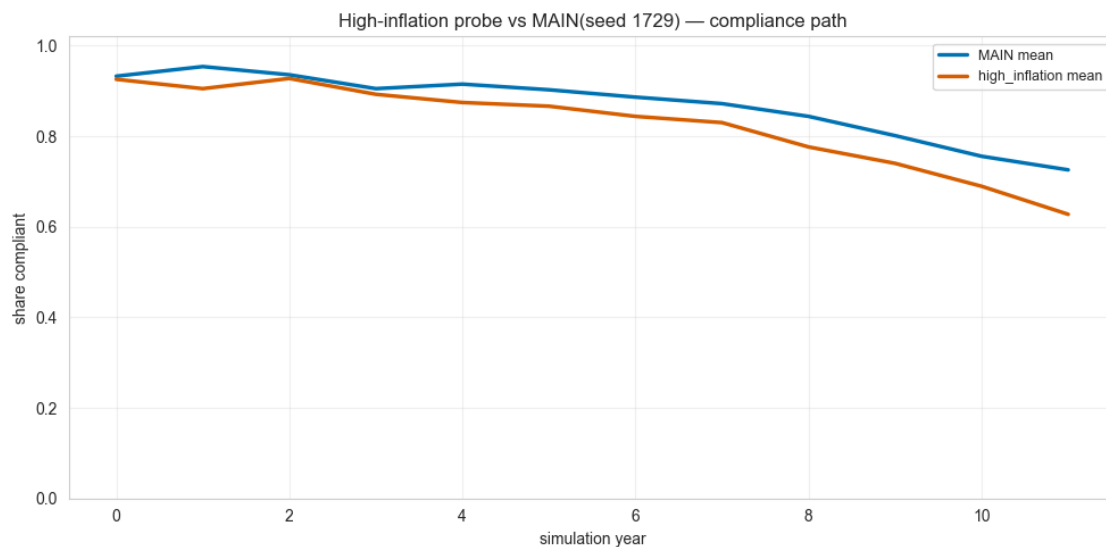
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\2486335180.py:39: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

plt.tight_layout(); plt.show()

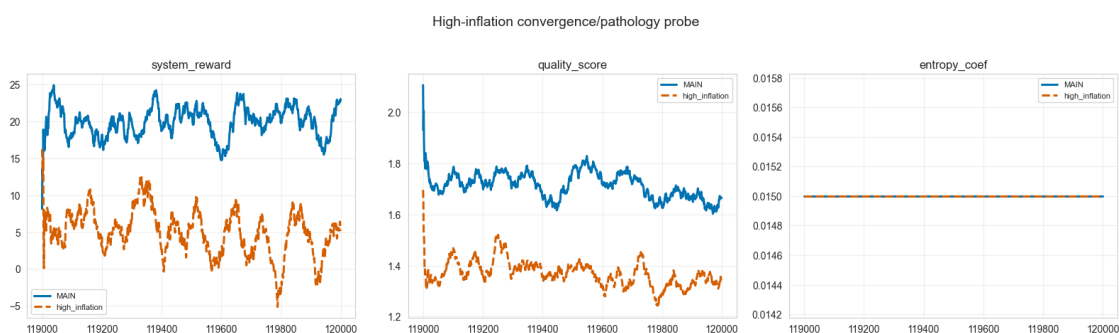
C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

fig.canvas.print_figure(bytes_io, **kw)





	variant	seed	mean_price	green_pct_year12	compliance_rate	
0	MAIN	1729	66.002251	73.370737	0.869062	
19	high_inflation	1729	78.897438	77.521002	0.824844	



Decision: YES - warrants a full 3-seed run.

Justification: compliance drop=0.044, quality drop=0.326, budget-gate clip jump=0.000.

1.13.1 13a. High-Inflation ESG Flow-Term Magnitude

This probe uses the same ESG-flow proxy as §12a to check whether rising real anchors make the ESG flow channel explode under the high-inflation single seed.

```
[24]: hi_reward_proxy = _agent_reward_proxy(['MAIN', 'high_inflation'], seeds=[1729])
if hi_reward_proxy.empty:
    print('No high-inflation reward-proxy data available.')
else:
    display(hi_reward_proxy.groupby(['variant', 'weighting'])[['cost_proxy',
    ↪ 'penalty', 'esg_flow_proxy', 'reward_base', 'reward_shaping']].mean().
    ↪ round(4))
    fig, ax = plt.subplots(figsize=(8, 4.5))
    sns.barplot(data=hi_reward_proxy, x='variant', y='esg_flow_proxy',
    ↪ hue='weighting', errorbar='sd', ax=ax)
    sns.stripplot(data=hi_reward_proxy, x='variant', y='esg_flow_proxy',
    ↪ hue='weighting', dodge=True,
    color='black', alpha=0.35, legend=False, ax=ax)
    ax.set_title('High-inflation probe: ESG flow-term proxy magnitude')
    ax.set_ylabel('ESG flow proxy')
    ax.grid(True, alpha=0.3, axis='y')
    plt.tight_layout(); plt.show()

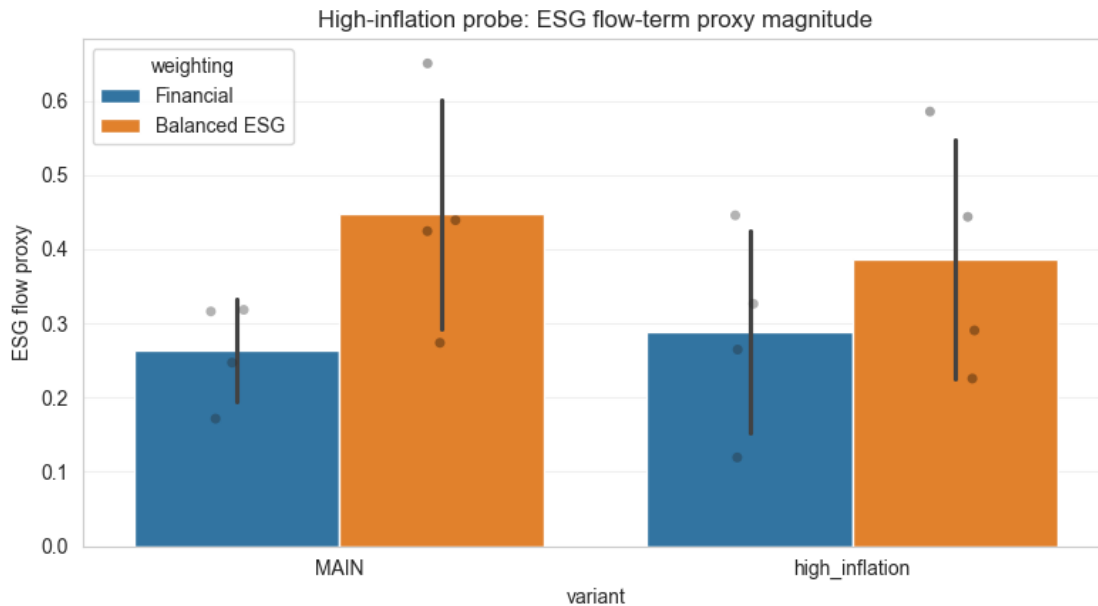
    piv = hi_reward_proxy.groupby('variant')['esg_flow_proxy'].mean()
    if {'MAIN', 'high_inflation'}.issubset(set(piv.index)):
        main_esg_flow = float(piv['MAIN'])
        ratio = float(piv['high_inflation']) / (main_esg_flow if main_esg_flow
    ↪ EPS else EPS)
        print(f'ESG-flow proxy ratio high_inflation / MAIN (seed 1729): {ratio:.
    ↪ 2f}x')
        if ratio > 2.0:
            print('FLAG: ESG-flow proxy more than doubles; inspect whether
    ↪ real-anchor scaling is dominating rewards.')
```

		cost_proxy	penalty	esg_flow_proxy	reward_base	↪
	↪reward_shaping					
variant	weighting					
MAIN	Balanced ESG	355.4207	5.9697	0.4468	0.4622	↪
↪	0.0433					
	Financial	272.4096	34.9705	0.2632	-0.1132	↪
↪	0.0421					
high_inflation	Balanced ESG	404.2931	11.4727	0.3862	0.2207	↪
↪	0.0498					
	Financial	359.6050	63.6888	0.2889	-0.2193	↪
↪	0.0468					

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\930182696.py:8: FutureWarning:

Setting a gradient palette using `color=` is deprecated and will be removed in v0.14.0. Set ``palette='dark:black'`` for the same effect.

```
sns.stripplot(data=hi_reward_proxy, x='variant', y='esg_flow_proxy',
hue='weighting', dodge=True,
```



ESG-flow proxy ratio high_inflation / MAIN (seed 1729): 0.95×

1.14 14. Headline Figures

```
[30]: # Fig 1: within-episode price path, all 4 regulatory regimes, with log2 y-scale.
import matplotlib.ticker as mticker

fig1_variants = ['MAIN', 'lrf_low_22pct', 'lrf_high_6pct',
↳ 'no_market_stability']
fig1_df = pd.concat(
    [_yearly_metric(v, price_metric, seeds=PAIRED_SEEDS) for v in
↳ fig1_variants],
    ignore_index=True
)

fig, ax = plt.subplots(figsize=(10, 5))
if fig1_df.empty:
    ax.text(0.5, 0.5, 'No year-level data', ha='center', va='center',
↳ transform=ax.transAxes)
else:
    for variant in fig1_variants:
```

```

sub = fig1_df[fig1_df['variant'] == variant]
if sub.empty:
    continue
by = sub.groupby(['seed', 'year'])['value'].mean().reset_index()

for seed, g in by.groupby('seed'):
    ax.plot(g['year'], g['value'], color=VARIANT_COLOR.get(variant),
    ↪alpha=0.25, lw=1)

agg = by.groupby('year')['value'].agg(['mean', 'std']).reset_index()
ax.plot(agg['year'], agg['mean'], color=VARIANT_COLOR.get(variant),
    ↪lw=2.4, label=f'{variant} mean')
    ax.fill_between(
        agg['year'],
        (agg['mean'] - agg['std'].fillna(0)).clip(lower=EPS),
        agg['mean'] + agg['std'].fillna(0),
        color=VARIANT_COLOR.get(variant),
        alpha=0.15
    )

# log base 2, but tick labels show real prices (not 2^k notation)
ax.set_yscale('log', base=2)
ax.yaxis.set_major_locator(mticker.LogLocator(base=2))
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda y, _: f'{y:.0f}'))
ax.yaxis.set_minor_formatter(mticker.NullFormatter())

ax.set(
    title='Fig 1 - Regulatory regimes: within-episode price path with seed_
    ↪bands (log y-axis)',
    xlabel='simulation year',
    ylabel='€/tCO2 (real price)'
)
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

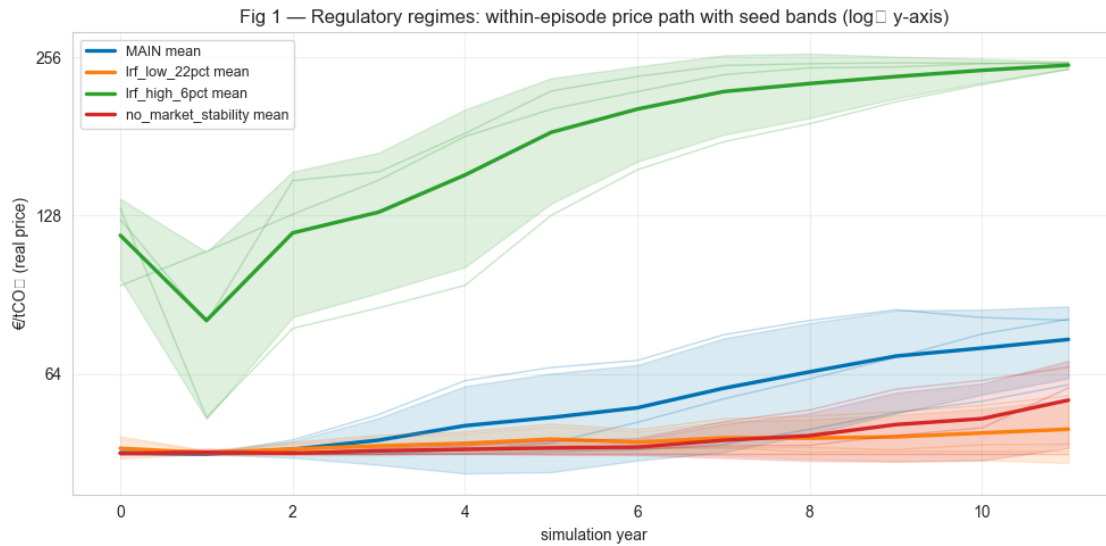
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\1333089454.py:46: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

```
plt.tight_layout()
```

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```



```
[31]: # Fig 2: year-12 outcomes bar chart, all variants.
fig2_metrics = ['green_pct_year12', 'compliance_rate', 'mean_price',
               ↪ 'penalty_frequency']
plot_variant_bars(fig2_metrics, 'Fig 2 - Year-12 / converged outcomes across',
               ↪ all variants', paired_only=False)
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:
FutureWarning:

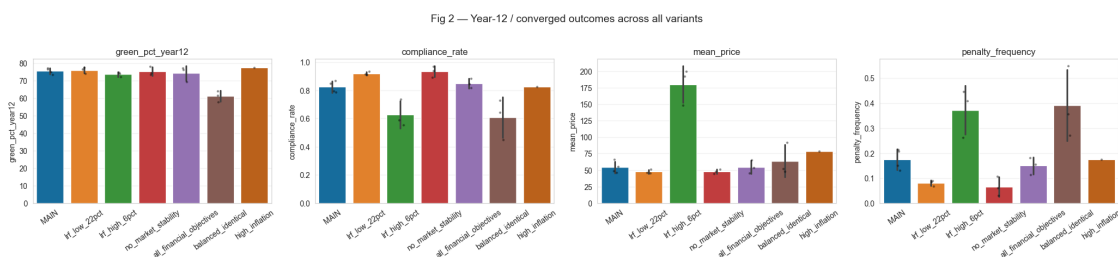
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same

effect.

```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
C:\Users\danie\AppData\Local\Temp\ipykernel_28284\460732173.py:11:
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='variant', y=metric, order=[v for v in VARIANT_ORDER if
v in set(df['variant'].astype(str))],
```



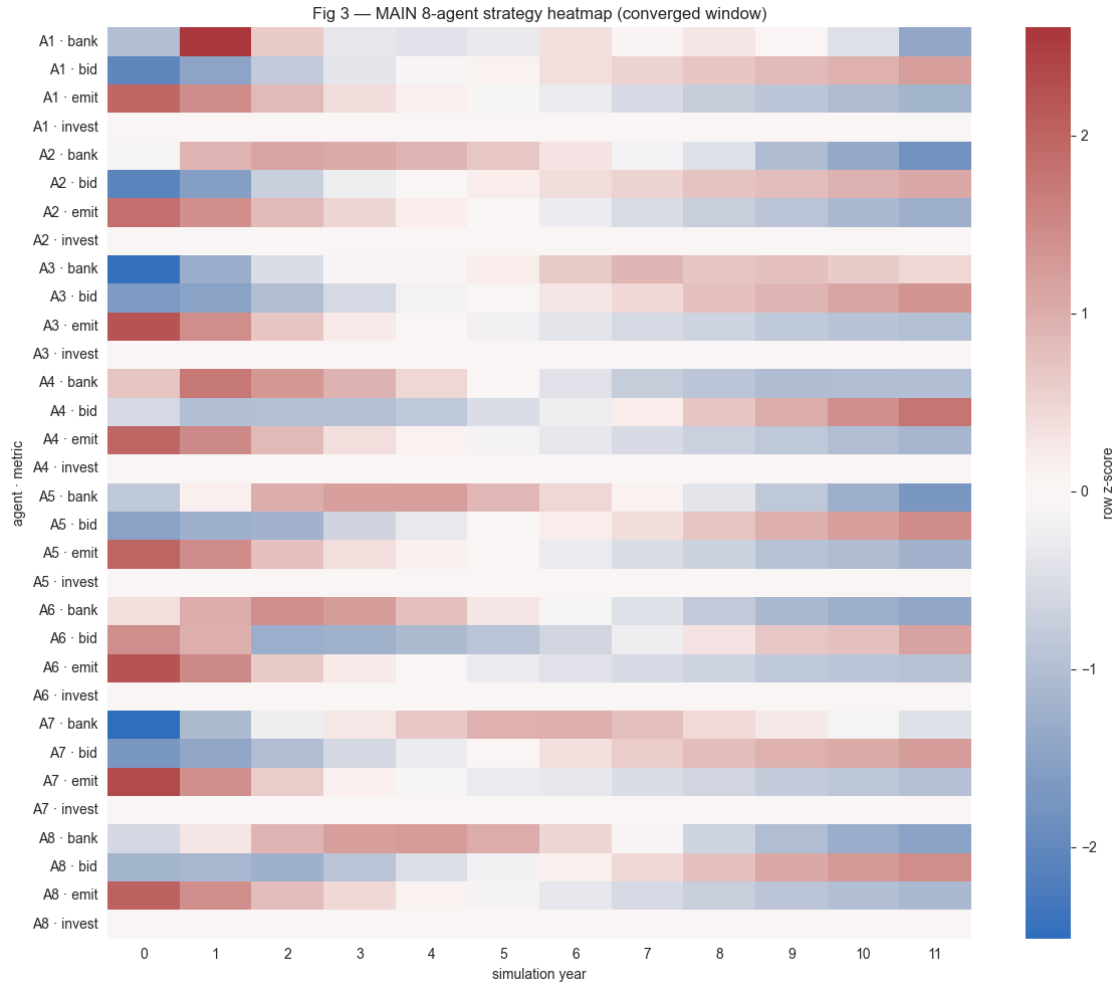
```
[32]: # Fig 3: 8-agent strategy heatmap for MAIN converged window (bid, invest, bank,
emit × year).
def main_strategy_heatmap(seed=None):
    main_keys = sorted([k for k in runs if k[0] == 'MAIN'])
    if seed is not None:
        main_keys = [k for k in main_keys if k[1] == seed]
    if not main_keys:
        print('No MAIN runs loaded.')
        return
    rows = []
    for k in main_keys:
        run = runs[k]
        yr = _tail_yr(run)
        if yr.empty:
            continue
        for i in range(N_AGENTS):
            for metric, prefix in [('bid', 'bid_price'), ('invest',
invest_frac_post_clip'), ('bank', 'bank_end'), ('emit', 'emissions')]:
                col = f'{prefix}_A{i+1}'
                if col not in yr.columns:
                    continue
                s = yr.groupby('year')[col].mean()
                for year, value in s.items():
```

```

        rows.append({'seed': k[1], 'agent': f'A{i+1}', 'metric': '
↪metric, 'year': year, 'value': value})
    df = pd.DataFrame(rows)
    if df.empty:
        print('No strategy heatmap data available.')
        return
    df = df.groupby(['agent', 'metric', 'year'])['value'].mean().reset_index()
    df['row'] = df['agent'] + ' · ' + df['metric']
    mat = df.pivot(index='row', columns='year', values='value')
    # Row-standardize so bid/invest/bank/emissions can share one heatmap.
    mat_z = mat.sub(mat.mean(axis=1), axis=0).div(mat.std(axis=1).replace(0, np.
↪nan), axis=0).fillna(0)
    fig, ax = plt.subplots(figsize=(12, 10))
    sns.heatmap(mat_z, cmap='vlag', center=0, ax=ax, cbar_kws={'label': 'row_
↪z-score'})
    ax.set_title('Fig 3 - MAIN 8-agent strategy heatmap (converged window)')
    ax.set_xlabel('simulation year'); ax.set_ylabel('agent · metric')
    plt.tight_layout(); plt.show()

main_strategy_heatmap()

```

```
[33]: # Fig 4: Sub-RQ 3 panel - fin vs ESG within MAIN + MAIN/all-fin/balanced
      ↪market-level deltas.
fig, axes = plt.subplots(1, 3, figsize=(17, 4.8))
if not main_agents.empty:
    sns.barplot(data=main_agents, x='weighting', y='green_frac', errorbar='sd',
    ↪ax=axes[0])
    axes[0].set_title('Within MAIN: green share by objective')
    sns.barplot(data=main_agents, x='weighting', y='bid_price', errorbar='sd',
    ↪ax=axes[1])
    axes[1].set_title('Within MAIN: bid price by objective')
else:
    axes[0].text(0.5, 0.5, 'No MAIN agent data', ha='center', va='center',
    ↪transform=axes[0].transAxes)
    axes[1].text(0.5, 0.5, 'No MAIN agent data', ha='center', va='center',
    ↪transform=axes[1].transAxes)
```

```

market = summary_df[summary_df['variant'].astype(str).isin(['MAIN',
    ↪ 'all_financial_objectives', 'balanced_identical'])]
sns.barplot(data=market, x='variant', y='green_pct_year12', errorbar='sd',
    ↪ order=['MAIN', 'all_financial_objectives', 'balanced_identical'],
    ↪ ax=axes[2], palette=VARIANT_COLOR)
axes[2].set_title('Market-level green% deltas')
for ax in axes:
    ax.tick_params(axis='x', rotation=25); ax.grid(True, alpha=0.3, axis='y')
plt.suptitle('Fig 4 - Sub-RQ 3: objectives, strategies, and aggregate
    ↪ outcomes', fontsize=14, y=1.03)
plt.tight_layout(); plt.show()

```

C:\Users\danie\AppData\Local\Temp\ipykernel_28284\1364028478.py:12:
FutureWarning:

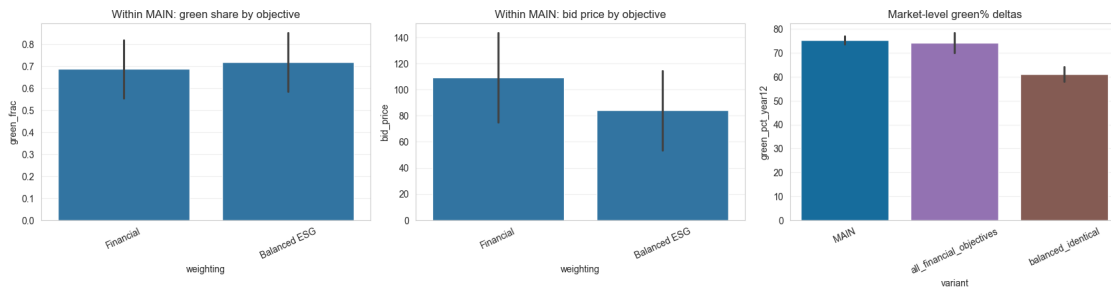
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.barplot(data=market, x='variant', y='green_pct_year12', errorbar='sd',
order=['MAIN', 'all_financial_objectives', 'balanced_identical'], ax=axes[2],
palette=VARIANT_COLOR)

```

Fig 4 — Sub-RQ 3: objectives, strategies, and aggregate outcomes



```

[34]: # Fig 5: convergence triptych for MAIN (reward, KL schedule/proxy, entropy).
main_runs = {k: v for k, v in runs.items() if k[0] == 'MAIN'}
fig, axes = plt.subplots(1, 3, figsize=(18, 4.8))

for (variant, seed), run in main_runs.items():
    ep = run['ep_df']
    if ep.empty:
        continue
    x = ep['episode'] if 'episode' in ep.columns else np.arange(len(ep))
    rew_cols = agent_cols(ep, 'reward')
    if rew_cols:

```

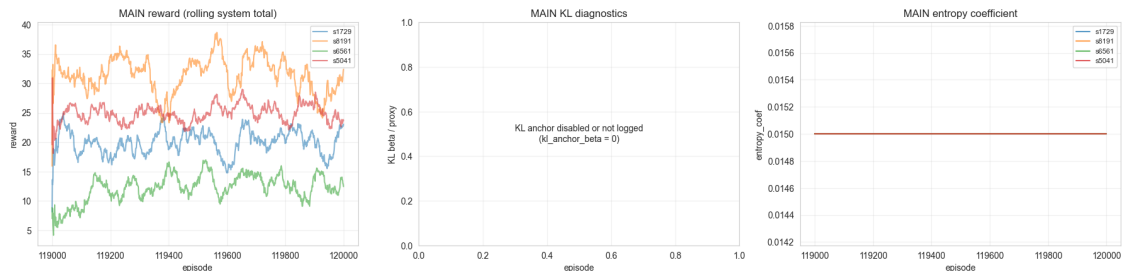
```

        axes[0].plot(x, ep[rewards_cols].sum(axis=1).rolling(ROLL, min_periods=1).
        ↪mean(), alpha=0.55, label=f's{seed}')
        if 'entropy_coef' in ep.columns:
            axes[2].plot(x, ep['entropy_coef'], alpha=0.75, label=f's{seed}')
axes[0].set_title('MAIN reward (rolling system total)'); axes[0].
        ↪set_xlabel('episode'); axes[0].set_ylabel('reward')

# KL beta is not written to training_log; reconstruct schedule from config if
        ↪enabled.
if main_runs:
    ref_run = next(iter(main_runs.values()))
    cfg = ref_run['config']
    ep_len = max((r.get('n_total_episodes', len(r['ep_df'])) for r in main_runs.
    ↪values()), default=0)
    beta0 = float(cfg.get('ppo', {}).get('kl_anchor_beta', 0.0))
    n_eps = int(cfg.get('simulation', {}).get('n_episodes', ep_len or 1))
    decay_raw = int(cfg.get('ppo', {}).get('kl_anchor_decay_episodes', 0) or 0)
    decay = decay_raw if decay_raw > 0 else int(0.50 * n_eps)
    xs = np.arange(ep_len)
    if beta0 > 0 and ep_len > 0:
        beta = beta0 * np.maximum(0, 1 - xs / max(decay, 1))
        axes[1].plot(xs, beta, color='purple', lw=2)
        axes[1].set_title('MAIN KL-anchor beta schedule (reconstructed)')
    else:
        axes[1].text(0.5, 0.5, 'KL anchor disabled or not
        ↪logged\n(kl_anchor_beta = 0)', ha='center', va='center', transform=axes[1].
        ↪transAxes)
        axes[1].set_title('MAIN KL diagnostics')
axes[1].set_xlabel('episode'); axes[1].set_ylabel('KL beta / proxy')
axes[2].set_title('MAIN entropy coefficient'); axes[2].set_xlabel('episode');
        ↪axes[2].set_ylabel('entropy_coef')
for ax in axes:
    ax.grid(True, alpha=0.3)
    if ax.get_legend_handles_labels()[0]:
        ax.legend(fontsize=8)
plt.suptitle('Fig 5 - MAIN convergence triptych', fontsize=14, y=1.03)
plt.tight_layout(); plt.show()

```

Fig 5 — MAIN convergence triptych



1.15 15. Research-Question Synthesis

Use this final section after reviewing the generated tables/figures.

1.15.1 RQ1 — Simulation credibility

Evidence to cite:

- **MAIN convergence triptych** (Fig 5): reward trajectory, entropy schedule, and KL-anchor status.
- **Cross-seed bands** in Fig 1 and summary-table standard deviations: whether seed noise is smaller than cross-variant differences.
- **quality_score**, **Q_compliance**, and **Q_volatility** in the overview table.

Suggested claim format:

The model is credible for sensitivity analysis if MAIN shows stable converged-window reward/quality, high compliance, bounded price volatility, and cross-seed variation smaller than the lrf/MSR/objective variant deltas.

1.15.2 RQ2 — Regulatory-mechanism sensitivity

Evidence to cite:

- **LRF low vs MAIN:** floor-touch frequency, low-TNAC scarcity timing, price path, and lower/higher green investment timing.
- **LRF high vs MAIN:** ceiling-touch frequency, earlier scarcity, carry-forward/penalty stress, and green investment pull-forward.
- **Cross-LRF elasticity:** $\Delta \text{green\%} / \Delta \text{pp LRF table}$ quantifies decarbonization responsiveness.
- **MSR-off:** if MAIN has zero MSR events, explicitly state the comparison is null for MSR mechanics; otherwise use TNAC, unsold volume, banking, and price variance to evaluate the stabilizer.

Suggested claim format:

Regulatory levers matter if LRF/MSR variants move both aggregate outcomes (price, compliance, green%) and mechanisms (TNAC, banking, unsold volume, carry-forward), not merely the headline price.

1.15.3 RQ3 — Strategic behaviour and aggregation

Evidence to cite:

- **MAIN decomposition:** financial vs ESG agents on bidding, investment, banking, secondary-market direction, and reward proxies.
- **all_financial_objectives:** whether removing ESG collapses strategy diversity or whether carbon-cost NPV still drives green investment.
- **balanced_identical:** whether identical endowments still diverge by reward objective; this isolates preference from initial asset mix.

- **Fig 3 heatmap:** year-by-year agent strategy structure in the converged MAIN market.

Suggested claim format:

Objective weights shape strategy if ESG agents differ systematically from financial agents inside MAIN and if the all-financial / balanced-identical counterfactuals alter the spread of bids, investments, banks, and secondary-market roles.

1.15.4 High-inflation probe decision

Use the automated single-seed decision line as a screening rule, then add qualitative judgment from the price/compliance/convergence plots. A full 3-seed run is warranted only if the high-inflation seed shows a clear compliance, budget-gate, volatility, or convergence break relative to MAIN seed 1729.